

Módulo 2 - Teoría

Curso Spring Boot 2026

Índice

Módulo 2 - Creación y estructura de proyectos Spring Boot	6
Propósito del módulo	6
Punto 2.1 - Capas de la aplicación: Controlador	6
Objetivos de aprendizaje	6
Bloque 1 - Qué Es Un Controlador YCuál Es Su Responsabilidad	6
1.1 Definición y rol del controlador	6
1.2 El controlador como traductor HTTP-Java	7
1.3 Lo que el controlador NO debe hacer	7
Pregunta	7
Respuesta razonada	7
Bloque 2 - Anotaciones De Mapeo En Profundidad	8
2.1 @Controller vs @RestController	8
2.2 @RequestMapping a nivel de clase	8
2.3 Anotaciones de método	8
Pregunta	9
Respuesta razonada	9
Bloque 3 - Extracción De Datos De La Petición	9
3.1 @PathVariable	9
3.2 @RequestParam	10
3.3 @RequestBody y @RequestHeader	10
Pregunta	11
Respuesta razonada	11
Bloque 4 - Construcción De La Respuesta	11
4.1 ResponseEntity	11
4.2 Códigos de estado y cabeceras	12
4.3 Serialización automática	12
Pregunta	12
Respuesta razonada	12
Bloque 5 - Anti-Patrones Y Buenas Prácticas	13
5.1 Controlador gordo	13
5.2 Controlador que accede al repositorio	13
5.3 Buenas prácticas en controladores	14
Pregunta	14
Respuesta razonada	14
Resumen de la teoría	14
Punto 2.2 - Capas de la aplicación: Servicio	14
Objetivos de aprendizaje	14
Bloque 1 - Qué Es Un Servicio YCuál Es Su Responsabilidad	15
1.1 Definición y rol del servicio	15
1.2 El servicio como orquestador	15
1.3 Lo que el servicio NO debe hacer	15
Pregunta	16
Respuesta razonada	16

Bloque 2 - La Anotación @Service	16
2.1 @Service, @Component, @Repository, @Controller	16
2.2 Cómo Spring detecta y registra los servicios	16
2.3 El ciclo de vida de un bean	17
Pregunta	17
Respuesta razonada	17
Bloque 3 - Inyección De Dependencias Por Constructor	17
3.1 Qué es la inyección de dependencias	17
3.2 Inyección por constructor vs por campo	17
3.3 Ventajas de la inyección por constructor en la práctica	18
Pregunta	18
Respuesta razonada	18
Bloque 4 - Transformación Dto ↔ Entidad	19
4.1 Por qué separar DTO y entidad	19
4.2 El patrón toDTO / toEntity	19
4.3 Dónde se transforma	19
Pregunta	20
Respuesta razonada	20
Bloque 5 - Excepciones De Negocio	20
5.1 Qué son y por qué se lanzan	20
5.2 Crear una excepción de negocio	20
5.3 Manejo con @ExceptionHandler (introducción)	21
Pregunta	21
Respuesta razonada	21
Resumen de la teoría	21
Punto 2.3 - Capas de la aplicación: Repositorio	21
Objetivos de aprendizaje	21
Bloque 1 - Qué es un repositorio y cuál es su responsabilidad	22
1.1 El repositorio es la frontera con los datos	22
1.2 Abstracción del almacenamiento	23
1.3 Lo que el repositorio no debe hacer	23
Pregunta	23
Respuesta razonada	23
Bloque 2 - @Repository y los estereotipos de Spring	23
2.1 @Repository frente a @Service y @Component	23
2.2 Detección automática	24
2.3 ¿Interfaz o clase?	24
Pregunta	25
Respuesta razonada	25
Bloque 3 - Diseño de un repositorio en memoria	25
3.1 Elegir la estructura de datos	25
3.2 ¿Por qué ConcurrentHashMap?	25
3.3 Operaciones del repositorio	25
3.4 Optional<T> en resultados únicos	26
3.5 No exponer el almacenamiento interno	26
Pregunta	27
Respuesta razonada	27
Bloque 4 - El servicio entre controlador y repositorio	27
4.1 El servicio deja de almacenar datos	27
4.2 Regla de DNI duplicado	27
4.3 Generación de identificadores: actualización respecto a la fuente	28
4.4 DTO y entidad: separación conceptual	28
4.5 Flujo completo	28
Pregunta	29
Respuesta razonada	29
Bloque 5 - Buenas prácticas, inmutabilidad y concurrencia	29
5.1 Buenas prácticas de diseño	29

5.2 Inmutabilidad observable	29
5.3 Concurrencia	30
5.4 Errores comunes	30
5.5 Cuando sirve un repositorio en memoria	30
Pregunta	30
Respuesta razonada	31
Resumen del punto 2.3	31
Punto 2.4 - Inyección de dependencias y configuración	31
Objetivos de aprendizaje	31
Bloque 1 - Inversión de control e inyección de dependencias	32
1.1 El problema de construir dependencias con new	32
1.2 Inversión de control	32
1.3 Inyección por constructor	32
1.4 Inyección por campo	33
Pregunta	33
Respuesta razonada	33
Bloque 2 - Estereotipos y detección de componentes	33
2.1 Los principales estereotipos	33
2.2 Component scanning	34
2.3 Qué ocurre cuando falta un bean	34
Pregunta	34
Respuesta razonada	34
Bloque 3 - @Configuration y @Bean	34
3.1 Cuando no basta con estereotipos	34
3.2 @Bean frente a @Component	35
3.3 ObjectMapper y Spring Boot	35
Pregunta	35
Respuesta razonada	35
Bloque 4 - CommandLineRunner y lógica de arranque	36
4.1 Qué es CommandLineRunner	36
4.2 Por qué sacar los datos del constructor del repositorio	36
4.3 Dependencias como parámetros del bean	36
Pregunta	37
Respuesta razonada	37
Bloque 5 - Configuración externalizada y perfiles	37
5.1 Por qué sacar valores del código	37
5.2 @Value	37
5.3 No exponer secretos	37
5.4 Perfiles	38
5.5 Orden de precedencia	38
Pregunta	38
Respuesta razonada	38
Resumen del punto 2.4	38
Profundización: ciclo de vida, formas de inyección y orden de arranque	39
Las tres formas de inyección	39
Ciclo de vida de un bean	39
Múltiples implementaciones	40
Varios CommandLineRunner	40
Más ejemplos de application.properties	40
Perfil de test	40
Singleton no significa "global mutable sin cuidado"	41
Nombre de los beans	41
@Component frente a @Bean: ejemplo completo	41
Qué significa que @Configuration sea especial	42
Configuración base y sobrescritura por entorno	42
Diagnóstico del grafo de dependencias	42
Una última regla práctica sobre configuración	42

Punto 2.5 - Introducción al testing	43
Objetivos de aprendizaje	43
Bloque 1 - Por qué testear	43
1.1 Un test es una especificación ejecutable	43
1.2 La pirámide de tests	43
1.3 Qué probar en cada capa	44
Pregunta	44
Respuesta razonada	44
Bloque 2 - JUnit 5	44
2.1 JUnit Jupiter	44
2.2 Ciclo de vida	44
2.3 Aserciones	45
2.4 Tests independientes	45
Pregunta	45
Respuesta razonada	45
Bloque 3 - Mockito	45
3.1 Por qué usamos mocks	45
3.2 Stubbing	45
3.3 Verificación de interacciones	46
3.4 No sobreverificar	46
Pregunta	46
Respuesta razonada	46
Bloque 4 - Tests unitarios del servicio	46
4.1 Estructura Arrange-Act-Assert	46
4.2 Consulta no encontrada	47
4.3 Regla de DNI duplicado	47
4.4 Creación correcta y evolución respecto a la fuente	47
Pregunta	47
Respuesta razonada	47
Bloque 5 - Tests web con @WebMvcTest y MockMvc	48
5.1 Slice test	48
5.2 MockMvc	48
5.3 @MockBean de la fuente y @MockitoBean en 2026	48
5.4 GET existente y 404	48
5.5 POST y Location	49
Pregunta	49
Respuesta razonada	49
Resumen del punto 2.5	49
Profundización: tipos de test, rojo-verde-refactor y buenas prácticas	49
Tipos de test	49
El ciclo rojo-verde-refactor	50
Buenas prácticas en tests de servicio	50
Casos de éxito y error	50
thenAnswer	51
Verificación con argumentos	51
Más verificaciones de MockMvc	51
Anotaciones y aserciones de JUnit 5 en conjunto	51
Mock, stub, fake y spy	52
Tests del repositorio en memoria	53
Test de copia defensiva	53
Test de concurrencia determinista	54
Qué carga @WebMvcTest	54
Test de JSON mal formado	54
Test de media type no soportado	54
Qué hace que un test sea mantenible	55
Datos de prueba expresivos	55
Probar excepciones sin capturarlas manualmente	55

@SpringBootTest no es “mejor” por cargar más	56
Pruebas parametrizadas	56
Organización de nombres de tests	56
El valor de ejecutar la suite completa	56
Fallos útiles y mensajes de aserción	56
Evitar tests que sólo duplican implementación	57
Regla final de la suite	57
Punto 2.6 - Buenas prácticas de organización, calidad y documentación	57
Objetivos de aprendizaje	57
Bloque 1 - Convenciones de nomenclatura	57
1.1 Los nombres forman parte del diseño	57
1.2 Nombres orientados a intención	58
1.3 Nombres de tests	58
Pregunta	58
Respuesta razonada	58
Bloque 2 - Organización de paquetes	59
2.1 Organización por capas	59
2.2 Organización por funcionalidad	59
2.3 Organización híbrida	59
2.4 Qué criterio usaremos en el curso	60
Pregunta	60
Respuesta razonada	60
Bloque 3 - Estructura interna de una clase	60
3.1 Orden consistente	60
3.2 Tamaño y responsabilidad	61
3.3 Visibilidad mínima necesaria	61
3.4 Repositorio hoy y en Spring Data JPA	61
Pregunta	61
Respuesta razonada	61
Bloque 4 - Herramientas de calidad	61
4.1 Checkstyle	61
4.2 SpotBugs	62
4.3 SonarQube	62
4.4 Calidad en CI	62
Pregunta	62
Respuesta razonada	62
Bloque 5 - Documentación: Javadoc, README y OpenAPI	63
5.1 Javadoc	63
5.2 Generar documentación Java	63
5.3 README	63
5.4 OpenAPI	64
5.5 Enriquecer el contrato	64
Pregunta	64
Respuesta razonada	64
Resumen del punto 2.6	64
Profundización: paquetes, comentarios y análisis estático	65
Convenciones de paquetes	65
Buenas prácticas específicas en controladores	65
Buenas prácticas específicas en servicios	65
Buenas prácticas específicas en repositorios	65
Comentarios en el código	66
SpotBugs con más detalle	66
SonarQube, cobertura y deuda técnica	66
Herramientas en CI	67
Convenciones concretas para tipos Java	67
Organización híbrida y paquetes comunes	67
Estructura interna: ejemplo completo	68

Comentarios de decisión	68
Configuraciones predefinidas de Checkstyle	68
Checkstyle no es un formateador	68
SpotBugs y falsos positivos	69
Cobertura no equivale a calidad	69
README como interfaz del proyecto	69
Documentación generada y documentación escrita	69
Calidad como conjunto de señales	69
Definición de terminado para calidad	70
Automatizar sin convertir la herramienta en el objetivo	70

Módulo 2 - Creación y estructura de proyectos Spring Boot

Propósito del módulo

M1 terminó con una API REST funcional: conocemos HTTP, JSON, Jackson, diseño de recursos y un CRUD en memoria. M2 cambia el foco. Ya no basta con que el código responda correctamente: empezamos a organizarlo para que pueda crecer sin convertir el controlador en el lugar donde ocurre todo.

El módulo recorre progresivamente la arquitectura en capas. Formalizamos la responsabilidad del **controlador**, movemos la lógica a un **servicio**, aislamos el acceso a datos en un **repositorio**, estudiamos cómo Spring conecta esas piezas mediante inversión de control e inyección de dependencias, aprendemos a probarlas y terminamos con organización, documentación y herramientas de calidad.

Baseline del curso: Java 17, Maven Wrapper 3.9.16 y Spring Boot 3.5.16.

Punto 2.1 - Capas de la aplicación: Controlador

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar con precisión cuál es la responsabilidad de un controlador.
2. Diferenciar @Controller de @RestController y saber cuándo usar cada uno.
3. Usar correctamente las anotaciones de mapeo a nivel de clase y de método.
4. Extraer datos de la petición con @PathVariable, @RequestParam, @RequestBody y @RequestHeader.
5. Construir respuestas HTTP con ResponseEntity, controlando código de estado y cabeceras.
6. Identificar y corregir los anti-patrones más comunes en controladores.
7. Refactorizar un controlador para que sea fino y sin lógica de negocio.

Nota 2026 - contrato acumulativo. Los ejemplos de este punto se estudian sobre el CRUD ya construido en M1. La implementación final conserva filtro curso, orden sort, paginación page/size, generación de ID por máximo numérico + 1, 201 Created con Location, /info-peticion, /promocionar y los enlaces del reto. Los helpers internos son un paso pedagógico: en 2.2 la lógica abandona definitivamente el controlador.

Bloque 1 - Qué Es Un Controlador YCuál Es Su Responsabilidad

1.1 Definición y rol del controlador

Un controlador es la puerta de entrada de una aplicación Spring Boot. Es la clase que recibe las peticiones HTTP, las interpreta y decide qué hacer con ellas. Cuando un cliente (navegador, app móvil, otro sistema) envía una petición a una URL, el controlador es el primero en enterarse. Su nombre lo

dice todo: controla el flujo de la petición. No ejecuta la lógica de negocio, no accede a la base de datos, no calcula nada complejo. Su trabajo es traducir el mundo HTTP al mundo Java y viceversa. Recibe una petición HTTP, extrae los datos, se los pasa a quien sabe procesarlos, recibe el resultado y construye una respuesta HTTP. Esa separación es fundamental. Si el controlador hiciera todo, tendríamos un código acoplado, difícil de testear y difícil de mantener. Al limitar su responsabilidad a la capa HTTP, conseguimos que el resto de la aplicación pueda evolucionar sin tocarlo. En Spring Boot, un controlador es una clase anotada con `@RestController` (o `@Controller`, que veremos en el siguiente bloque) que contiene métodos anotados con `@GetMapping`, `@PostMapping`, etc. Cada método se corresponde con un endpoint: una combinación de URL y método HTTP que el cliente puede invocar.

1.2 El controlador como traductor HTTP-Java

Cuando un cliente envía una petición HTTP, llega al servidor en forma de texto estructurado: una línea de petición, cabeceras y, a veces, un cuerpo. El controlador tiene que convertir todo eso en objetos Java que el resto de la aplicación pueda entender. Esa traducción ocurre en varias direcciones: De HTTP a Java. El controlador extrae el ID de la URL, los parámetros de la query, las cabeceras y el cuerpo JSON. Los

- convierte en tipos Java: String, Integer, AlumnoDTO, etc. De Java a HTTP. El controlador recibe un objeto Java como resultado y construye una respuesta HTTP: código de estado,
- cabeceras y cuerpo (normalmente JSON). El controlador no sabe de dónde vienen los datos ni adónde van. No sabe si el cliente es un navegador o una app móvil. No sabe si el resultado viene de una base de datos o de un servicio externo. Solo sabe que ha recibido una petición HTTP y que debe devolver una respuesta HTTP. Esa ignorancia es una virtud: permite que el controlador sea reemplazable y testeable. Puedes cambiar la lógica de negocio sin tocar el controlador, y puedes probar el controlador sin arrancar la base de datos.

1.3 Lo que el controlador NO debe hacer

Tan importante como saber qué hace un controlador es saber qué no debe hacer. Estos son los límites: No debe contener lógica de negocio. Las reglas de negocio (validaciones complejas, cálculos, decisiones) van en la capa de servicio. El controlador solo delega. No debe acceder directamente a la base de datos. El acceso a datos va en la capa de repositorio. El controlador no debe inyectar un repositorio ni ejecutar consultas. No debe transformar entidades a DTOs. Esa transformación es responsabilidad del servicio. El controlador recibe DTOs y devuelve DTOs; nunca ve entidades. No debe gestionar transacciones. La gestión de transacciones va en el servicio. No debe construir la respuesta manualmente. No debe usar `StringBuilder` para construir JSON. Devuelve objetos y deja que Jackson los serialice. No debe capturar todas las excepciones. El manejo de errores se centraliza en un manejador global. El controlador no debe tener try-catch en cada método. No debe tener estado. Los controladores son singletons: una sola instancia atiende todas las peticiones. Si tienen campos mutables, dos peticiones concurrentes pueden corromper el estado. Los controladores deben ser sin estado: toda la información va en parámetros.

Pregunta

¿Por qué un controlador con estado es peligroso en Spring Boot? ¿Qué pasaría si dos peticiones concurrentes modifican el mismo campo?

Respuesta razonada

Porque los controladores son normalmente beans singleton y pueden atender varias peticiones a la vez. Un campo mutable compartido puede mezclar datos entre peticiones; el estado de una petición debe permanecer en parámetros y variables locales.

Bloque 2 - Anotaciones De Mapeo En Profundidad

2.1 @Controller vs @RestController

Spring ofrece dos anotaciones para marcar una clase como controlador: `@Controller` y `@RestController`. No son intercambiables: cada una tiene su propósito. `@Controller` es la anotación original de Spring MVC. Marca la clase como controlador, pero no añade `@ResponseBody` a los métodos. Eso significa que, por defecto, los métodos devuelven el nombre de una vista (una plantilla HTML) que Spring MVC renderiza. Se usa en aplicaciones web tradicionales con renderizado en servidor (Thymeleaf, JSP, FreeMarker). `@RestController` es una especialización de `@Controller` que añade `@ResponseBody` a todos los métodos de la clase. Eso significa que los métodos devuelven directamente el cuerpo de la respuesta, no el nombre de una vista. Se usa en APIs REST, donde las respuestas son JSON o XML. La diferencia es importante porque afecta al comportamiento por defecto. Si usas `@Controller` en una API REST, Spring MVC intentará buscar una vista HTML y devolverá un error 404 o 500. Si usas `@RestController` en una aplicación web tradicional, Spring MVC escribirá el resultado del método como cuerpo de la respuesta, sin renderizar ninguna vista. Para este curso, y para cualquier API REST moderna, usaremos siempre `@RestController`. La regla es simple: API REST → `@RestController`; aplicación web con vistas → `@Controller`.

2.2 @RequestMapping a nivel de clase

La anotación `@RequestMapping` se puede poner a nivel de clase o a nivel de método. A nivel de clase, define un prefijo común para todas las rutas de los métodos de esa clase. Por ejemplo:

```
@RestController
@RequestMapping("/api/v1/alumnos")
public class AlumnoController {
    @GetMapping
    public List<AlumnoDTO> listar() { ... }
    @GetMapping("/{id}")
    public ResponseEntity<AlumnoDTO> consultar(@PathVariable String id) { ... }
    @PostMapping
    public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) { ... }
}
```

Con `@RequestMapping("/api/v1/alumnos")` a nivel de clase, las rutas finales son: GET `/api/v1/alumnos` (el método `listar`).

- GET `/api/v1/alumnos/{id}` (el método `consultar`).
- POST `/api/v1/alumnos` (el método `crear`).
- Sin la anotación a nivel de clase, tendrías que repetir `/api/v1/alumnos` en cada método. Eso sería redundante y propenso a errores. Ponerlo a nivel de clase centraliza el prefijo y hace que el código sea más limpio. `@RequestMapping` también acepta otros atributos, como `consumes` (el tipo de contenido que acepta) y `produces` (el tipo de contenido que devuelve). Por ejemplo:

```
@RequestMapping(value = "/api/v1/alumnos", produces = "application/json")
```

Pero en la práctica, con `@RestController` y Jackson, no hace falta especificar `produces`: Spring Boot detecta que el cliente acepta JSON y serializa automáticamente.

2.3 Anotaciones de método

A nivel de método, se usan anotaciones específicas para cada método HTTP. Ya las conocemos del Módulo 1: `@GetMapping` para GET.

- `@PostMapping` para POST.
- `@PutMapping` para PUT.
- `@PatchMapping` para PATCH.
- `@DeleteMapping` para DELETE.

- Todas ellas son especializaciones de `@RequestMapping`. `@GetMapping("/{id}")` es equivalente a `@RequestMapping(value = "/{id}", method = RequestMethod.GET)`. Las especializadas son más concisas y más legibles. Estas anotaciones aceptan una ruta como argumento. La ruta puede ser: Vacía (sin argumento): mapea a la URL de la clase.
- Relativa (`"/{id}"`): se concatena con la URL de la clase.
- Absoluta (`"/otra/ruta"`): reemplaza la URL de la clase.
- En la práctica, se usan rutas relativas, porque el prefijo de la clase ya aporta el contexto. También aceptan path variables entre llaves (`{id}`). Esas variables se capturan con `@PathVariable` en los parámetros del método, como veremos en el siguiente bloque.

Pregunta

¿Por qué es mejor poner el prefijo de la URL a nivel de clase y no en cada método?

Respuesta razonada

Porque expresa una raíz común del recurso en un único lugar, reduce repetición y evita rutas inconsistentes entre métodos.

Bloque 3 - Extracción De Datos De La Petición

3.1 @PathVariable

La anotación `@PathVariable` extrae un valor de la ruta de la URL y lo inyecta como parámetro del método. Es la forma de capturar identificadores de recursos. Por ejemplo, en un método mapeado con `@GetMapping("/{id}")`:

```
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable String id) {
// ...
}
```

El `{id}` en la ruta se captura con `@PathVariable String id`. Si el cliente visita `/api/v1/alumnos/12345`, el parámetro `id` valdrá `"12345"`. El nombre de la variable en la ruta y el nombre del parámetro deben coincidir. Si no coinciden, se puede especificar explícitamente:

```
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable("id") String identificador) {
// ...
}
```

Aquí la variable de la ruta es `id`, pero el parámetro del método se llama `identificador`. La anotación `@PathVariable("id")` hace la conexión. `@PathVariable` puede capturar varios valores a la vez:

```
@GetMapping("/{id}/documentos/{docId}")
public ResponseEntity<DocumentoDTO> consultarDocumento(
    @PathVariable String id,
    @PathVariable String docId) {
// ...
}
```

Los path variables son obligatorios por defecto. Si la ruta no incluye el valor, Spring MVC no mapea la petición a ese método.

3.2 @RequestParam

La anotación @RequestParam extrae un valor de la query string de la URL y lo inyecta como parámetro del método. Es la forma de capturar filtros, opciones de paginación y otros parámetros opcionales. Por ejemplo:

```
@GetMapping
public List<AlumnoDTO> listar(@RequestParam(required = false) String curso) {
// ...
}
```

Si el cliente visita /api/v1/alumnos?curso=5º Primaria, el parámetro curso valdrá "5º Primaria". Si visita /api/v1/alumnos sin query, curso será null porque hemos puesto required = false. Por defecto, @RequestParam es obligatorio: si el cliente no envía el parámetro, Spring MVC devuelve un 400 Bad Request. Para hacerlo opcional, se usa required = false. Para darle un valor por defecto, se usa defaultValue:

```
@GetMapping
public List<AlumnoDTO> listar(
    @RequestParam(required = false) String curso,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size) {
// ...
}
```

Aquí curso es opcional (será null si no se envía), mientras que page y size tienen valores por defecto (0 y 20). Si el cliente no los envía, se usan esos valores. @RequestParam también puede capturar varios valores a la vez:

```
@GetMapping
public List<AlumnoDTO> listar(@RequestParam List<String> cursos) {
// ...
}
```

Si el cliente envía /api/v1/alumnos?cursos=5º&cursos=6º, la lista cursos contendrá ["5º", "6º"].

3.3 @RequestBody y @RequestHeader

La anotación @RequestBody captura el cuerpo de la petición y lo deserializa a un objeto Java. Es la forma de recibir datos en POST, PUT y PATCH.

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
// ...
}
```

Spring MVC detecta que el Content-Type es application/json y usa Jackson para deserializar el cuerpo al tipo indicado (AlumnoDTO). Si el cuerpo está mal formado, Spring MVC devuelve un 400 Bad Request. @RequestBody solo se puede usar en métodos que aceptan cuerpo (POST, PUT, PATCH). En GET y DELETE, no tiene sentido. La anotación @RequestHeader captura una cabecera de la petición y la inyecta como parámetro del método. Es útil para leer metadatos como el User-Agent, el Authorization o una cabecera personalizada.

```
@GetMapping
public List<AlumnoDTO> listar(
    @RequestHeader(value = "User-Agent", required = false) String userAgent) {
// ...
}
```

```
}
```

Si el cliente envía la cabecera User-Agent, el parámetro la contendrá. Si no, será null (porque required = false). @RequestHeader también se puede usar para leer varias cabeceras:

```
@GetMapping
public List<AlumnoDTO> listar(
    @RequestHeader Map<String, String> cabeceras) {
// ...
}
```

En este caso, cabeceras contendrá todas las cabeceras de la petición como un mapa.

Pregunta

¿Qué diferencia hay entre @PathVariable y @RequestParam? ¿Cuándo usarías cada uno?

Respuesta razonada

@PathVariable forma parte de la ruta que identifica o jerarquiza el recurso; @RequestParam suele expresar filtros, paginación u opciones de una consulta.

Bloque 4 - Construcción De La Respuesta

4.1 ResponseEntity

ResponseEntity es una clase de Spring que representa una respuesta HTTP completa: código de estado, cabeceras y cuerpo. Es la forma más flexible de construir respuestas en un controlador. Un método que devuelve ResponseEntity puede controlar: El código de estado (200 OK, 201 Created, 404 Not Found, etc.).

- Las cabeceras (Location, Content-Type, etc.).
- El cuerpo (el objeto que se serializa a JSON).
- Por ejemplo:

```
@GetMapping("/{id}")
public ResponseEntity<AlumnoDTO> consultar(@PathVariable String id) {
    Optional<AlumnoDTO> encontrado = buscarPorId(id);
    return encontrado
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

Aquí, si el alumno existe, se devuelve ResponseEntity.ok(alumno) con código 200 y el alumno en el cuerpo. Si no existe, se devuelve ResponseEntity.notFound().build() con código 404 y sin cuerpo. ResponseEntity ofrece métodos estáticos para construir respuestas comunes: ResponseEntity.ok(cuerpo) → 200 con cuerpo.

- ResponseEntity.created(uri).body(cuerpo) → 201 con Location y cuerpo.
- ResponseEntity.noContent().build() → 204 sin cuerpo.
- ResponseEntity.notFound().build() → 404 sin cuerpo.
- ResponseEntity.badRequest().body(cuerpo) → 400 con cuerpo.
- ResponseEntity.status(HttpStatus.CONFLICT).body(cuerpo) → 409 con cuerpo.
- También se puede construir manualmente:

```
return ResponseEntity
    .status(HttpStatus.CREATED)
    .header("X-Custom-Header", "valor")
    .body(alumno);
```

Esto crea una respuesta 201 con una cabecera personalizada y un cuerpo.

4.2 Códigos de estado y cabeceras

El código de estado es la forma que tiene el servidor de comunicar el resultado de la operación. Ya vimos en el Módulo 1 cuáles son los códigos correctos para cada operación. En el controlador, se eligen usando `ResponseEntity`. Además del código, `ResponseEntity` permite añadir cabeceras a la respuesta. Las más útiles son: `Location`: indica la URL del recurso creado (se usa en POST).

- `ETag`: identificador de versión del recurso (para caché).
- `Cache-Control`: directivas de caché.
- `X-*`: cabeceras personalizadas de la aplicación.
- Por ejemplo, en un POST que crea un recurso:

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    AlumnoDTO creado = guardar(dto);
    URI location = URI.create("/api/v1/alumnos/" + creado.getIdentificador());
    return ResponseEntity.created(location).body(creado);
}
```

`ResponseEntity.created(location)` construye un 201 con la cabecera `Location` apuntando al recurso creado. El cliente puede usar esa URL para consultar el recurso.

4.3 Serialización automática

Una de las ventajas de Spring Boot es que no tienes que serializar manualmente el cuerpo de la respuesta. Cuando devuelves un objeto Java (DTO, Map, List), Spring MVC detecta que el cliente acepta `application/json` y usa Jackson para serializarlo. Eso significa que: No escribes JSON a mano.

- No construyes strings con `StringBuilder`.
- No configuras el `Content-Type` manualmente (Spring MVC lo hace).
- Solo devuelves el objeto y Spring Boot se encarga del resto. La serialización respeta las anotaciones de Jackson que ya conocemos: `@JsonProperty`, `@JsonFormat`, `@JsonInclude`, `@JsonIgnore`. Si el método devuelve `String`, Spring MVC lo escribe como texto plano con `Content-Type: text/plain`. Si devuelve un `Map` o un `DTO`, lo serializa a JSON con `Content-Type: application/json`. Si devuelve `ResponseEntity`, respeta el código de estado y las cabeceras que hayas definido. La regla es simple: devuelve objetos, no strings. Deja que Spring Boot se encargue de la serialización.

Pregunta

¿Por qué es mejor devolver objetos en lugar de construir el JSON a mano?

Respuesta razonada

Porque Spring MVC y Jackson pueden serializarlos de forma consistente, respetando negociación de contenido y anotaciones. Construir JSON a mano duplica trabajo y facilita errores de escapado y formato.

Bloque 5 - Anti-Patrones Y Buenas Prácticas

5.1 Controlador gordo

El controlador gordo es el anti-patrón más común. Ocurre cuando el controlador acumula responsabilidades que no le corresponden: lógica de negocio, acceso a datos, validaciones complejas, transformaciones. Un controlador gordo suele tener métodos largos, con muchas líneas, que hacen varias cosas a la vez. Por ejemplo:

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    // Validar que el DNI no existe
    for (AlumnoDTO a : alumnos) {
        if (a.getDni().equals(dto.getDni())) {
            return ResponseEntity.status(HttpStatus.CONFLICT).build();
        }
    }

    // Asignar ID
    dto.setIdentificador(String.valueOf(alumnos.size() + 1));

    // Añadir a la lista
    alumnos.add(dto);

    // Devolver
    return ResponseEntity.status(HttpStatus.CREATED).body(dto);
}
```

Este método mezcla validación de negocio, generación de ID y persistencia. Si mañana cambia la regla del DNI, hay que tocar el controlador. Si mañana se añade una regla más, el método crece. La solución es delegar: mover toda esa lógica a una capa de servicio. El controlador se queda solo con la parte HTTP:

```
@PostMapping
public ResponseEntity<AlumnoDTO> crear(@RequestBody AlumnoDTO dto) {
    AlumnoDTO creado = servicio.crear(dto);
    return ResponseEntity.status(HttpStatus.CREATED).body(creado);
}
```

Mucho más limpio. El controlador no sabe cómo se valida ni cómo se genera el ID. Solo sabe que ha recibido una petición y que debe devolver una respuesta.

5.2 Controlador que accede al repositorio

Otro anti-patrón es que el controlador inyecte un repositorio y acceda directamente a la base de datos. Eso se salta la capa de servicio, donde debería estar la lógica de negocio.

```
@RestController
public class AlumnoController {
    @Autowired
    private AlumnoRepository repository; // Anti-patrón

    @GetMapping("/{id}")
    public ResponseEntity<Alumno> consultar(@PathVariable String id) {
        return repository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

Este controlador devuelve una entidad (Alumno), no un DTO. Eso expone la estructura de la base de datos. Además, no hay separación entre capas: si mañana cambia la lógica de negocio, hay que tocar el controlador. La solución es inyectar el servicio, no el repositorio. El servicio se encarga de acceder al repositorio y de transformar la entidad a DTO. El controlador solo ve DTOs.

5.3 Buenas prácticas en controladores

Además de evitar los anti-patrones, hay una serie de buenas prácticas que hacen que un controlador sea limpio y mantenible: Métodos cortos. Un método de controlador no debería superar las 5-10 líneas. Si es más largo, probablemente está haciendo cosas que no le corresponden. Inyección por constructor. En lugar de `@Autowired` en el campo, se inyectan las dependencias por constructor. Eso permite que los campos sean final y que el controlador sea inmutable. Sin estado. Los controladores no deben tener campos mutables. Toda la información va en parámetros. Sin try-catch. El manejo de errores se centraliza en un manejador global. El controlador no captura excepciones; las deja propagar. Sin lógica de negocio. Ya lo hemos dicho, pero conviene repetirlo: el controlador solo delega. DTOs, no entidades. El controlador nunca ve entidades JPA. Solo DTOs. Códigos de estado correctos. 200, 201, 204, 400, 404, 409. No todo es 200. Documentación con OpenAPI. Añadir anotaciones de Springdoc para que la API se documente automáticamente.

Pregunta

¿Qué ventaja tiene inyectar dependencias por constructor en lugar de por campo? ¿Qué permite que no permite la inyección por campo?

Respuesta razonada

La inyección por constructor hace visibles las dependencias obligatorias, permite campos final y facilita tests directos sin manipular campos privados ni arrancar Spring.

Resumen de la teoría

Controlador: puerta de entrada HTTP; traduce HTTP $\leftrightarrow$ Java; no tiene lógica de negocio.

- `@RestController`: para APIs REST; combina `@Controller` + `@ResponseBody`.
- `@Controller`: para aplicaciones web con vistas.
- `@RequestMapping` a nivel de clase: prefijo común para todas las rutas.
- Anotaciones de método: `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`.
- `@PathVariable`: extrae valores de la ruta.
- `@RequestParam`: extrae valores de la query.
- `@RequestBody`: deserializa el cuerpo a un objeto Java.
- `@RequestHeader`: extrae cabeceras.
- `ResponseEntity`: controla código de estado, cabeceras y cuerpo.
- Serialización automática: devolver objetos, no strings.
- Anti-patrones: controlador gordo, controlador que accede al repositorio.
- Buenas prácticas: métodos cortos, inyección por constructor, sin estado, sin try-catch, DTOs.

Punto 2.2 - Capas de la aplicación: Servicio

Objetivos de aprendizaje

Al finalizar esta sesión, el alumno será capaz de:

1. Explicar cuál es la responsabilidad de un servicio y por qué existe.
2. Diferenciar @Service de @Component, @Repository y @Controller.
3. Aplicar inyección de dependencias por constructor y entender por qué es la forma recomendada.
4. Transformar entre DTOs y entidades dentro del servicio.
5. Lanzar excepciones de negocio específicas y manejarlas con @ExceptionHandler.
6. Refactorizar un controlador para que delegue toda la lógica en un servicio.
7. Diagnosticar y resolver los errores más comunes al extraer lógica a un servicio.

Nota 2026 - separación progresiva. El servicio se introduce sin adelantar todavía la capa de repositorio. La lista en memoria vive temporalmente en `AlumnoService`; la regla de DNI duplicado se expresa con `NegocioException`, el controlador la traduce a `409 Conflict` y se mantiene la generación de ID segura heredada de M1. La separación DTO/entidad se explica aquí como patrón y se materializará cuando exista un modelo de persistencia distinto.

Bloque 1 - Qué Es Un Servicio YCuál Es Su Responsabilidad

1.1 Definición y rol del servicio

Un servicio es la clase donde vive la lógica de negocio de la aplicación. Es el corazón del back-end. Mientras que el controlador se ocupa de traducir HTTP a Java, el servicio se ocupa de decidir qué hacer con esos datos: validar reglas de negocio, calcular importes, coordinar operaciones, transformar datos y persistirlos. El servicio no sabe nada de HTTP. No conoce `ResponseEntity`, ni `@GetMapping`, ni códigos de estado. Tampoco sabe nada de la base de datos: no ejecuta consultas SQL ni conoce tablas. Su mundo es el de las reglas de negocio y los objetos de dominio. Cuando un controlador recibe una petición, no decide por sí mismo qué hacer. Delega en el servicio. El controlador dice: “me han pedido crear un alumno con estos datos”. El servicio dice: “voy a validar que el DNI no esté duplicado, asignar un identificador, guardar el alumno y devolverlo”. El controlador recibe el resultado y construye la respuesta HTTP. Esa separación es fundamental. Si el controlador tomara las decisiones, tendríamos un código acoplado: cualquier cambio en las reglas de negocio obligaría a tocar el controlador. Al mover las decisiones al servicio, el controlador se queda con una responsabilidad clara y el servicio con otra. Cada uno puede evolucionar por separado.

1.2 El servicio como orquestador

Además de aplicar reglas de negocio, el servicio actúa como orquestador. Cuando una operación requiere varias acciones, el servicio las coordina. Por ejemplo, crear un alumno puede requerir:

1. Validar que el DNI no esté duplicado.
2. Asignar un identificador único.
3. Guardar el alumno en la base de datos.
4. Notificar a otro servicio (por ejemplo, al sistema de matrículas).
5. Devolver el alumno creado.

El controlador no sabe nada de esos pasos. Solo sabe que ha llamado a `servicio.crear(dto)` y que ha recibido un resultado. El servicio es quien conoce la secuencia, las dependencias y las reglas. Esa orquestación es lo que hace que el servicio sea el lugar adecuado para la lógica de negocio. El controlador no debería saber cuántos pasos hay ni en qué orden se ejecutan. Solo debería saber que hay una operación llamada “crear alumno” y que devuelve un resultado.

1.3 Lo que el servicio NO debe hacer

Igual que con el controlador, conviene saber qué no debe hacer un servicio: No debe conocer HTTP. No debe importar `ResponseEntity`, `HttpStatus`, `@GetMapping` ni nada del paquete `org.springframework.web`. El servicio debe poder usarse desde un controlador REST, desde un batch,

desde un test, sin cambios. No debe construir respuestas HTTP. No debe devolver `ResponseEntity`. Devuelve objetos de dominio o DTOs; el controlador construye la respuesta. No debe acceder directamente a la base de datos. No debe usar `EntityManager` ni `JdbcTemplate`. El acceso a datos va en la capa de repositorio, que veremos en el siguiente punto. El servicio invoca al repositorio y recibe entidades. No debe capturar excepciones y devolver null. Si algo falla, debe lanzar una excepción. El controlador o un manejador global se encargarán de convertirla en una respuesta HTTP. No debe tener estado. Los servicios son singletons. Si tienen campos mutables, dos peticiones concurrentes pueden corromper el estado. Toda la información va en parámetros.

Pregunta

¿Por qué un servicio no debe conocer HTTP? ¿Qué ventaja tiene que pueda usarse desde un batch o un test sin cambios?

Respuesta razonada

Porque el servicio representa lógica de aplicación reutilizable. Si dependiera de `ResponseEntity` o códigos HTTP, quedaría acoplado a un único canal de entrada.

Bloque 2 - La Anotación @Service

2.1 @Service, @Component, @Repository, @Controller

Spring ofrece varias anotaciones para marcar clases como beans (componentes gestionados por el contenedor de Spring). Todas ellas son especializaciones de `@Component`, pero cada una tiene un significado semántico distinto: `@Component`: anotación genérica. Marca una clase como bean de Spring sin un rol específico. Se usa cuando la clase no encaja

- en ninguna de las otras categorías. `@Service`: marca una clase como servicio. Su rol es contener lógica de negocio.
- `@Repository`: marca una clase como repositorio. Su rol es el acceso a datos. Además, Spring añade traducción automática de
- excepciones de base de datos. `@Controller`: marca una clase como controlador MVC. Su rol es atender peticiones HTTP y devolver vistas.
- `@RestController`: especialización de `@Controller` que añade `@ResponseBody`. Su rol es atender peticiones HTTP y devolver el
- cuerpo directamente. Todas ellas registran la clase en el contenedor de Spring. La diferencia es semántica: indican a otros desarrolladores (y a las herramientas) cuál es el rol de la clase. Un `@Service` no debería hacer acceso a datos, y un `@Repository` no debería tener lógica de negocio. Las anotaciones comunican esa intención. En la práctica, podrías usar `@Component` para todo y funcionaría. Pero usar la anotación correcta hace que el código sea más legible y que las herramientas de análisis estático detecten mejor los problemas.

2.2 Cómo Spring detecta y registra los servicios

Cuando Spring Boot arranca, escanea el paquete raíz (el de la clase `@SpringBootApplication`) y todos sus subpaquetes buscando clases anotadas con `@Component` o sus especializaciones (`@Service`, `@Repository`, `@Controller`, `@RestController`). Para cada clase encontrada:

1. Crea una instancia (un bean).
2. Resuelve sus dependencias (inyecta lo que necesite).
3. La registra en el contexto de aplicación (el contenedor de beans).
4. La deja disponible para que otras clases la inyecten.

Por defecto, los beans son singletons: Spring crea una sola instancia y la reutiliza en todas las inyecciones. Eso significa que un `AlumnoService` se crea una vez y se comparte entre todos los controladores que lo necesiten. El escaneo se hace al arrancar. Si una clase no está en el paquete raíz ni en sus subpaquetes, Spring no la encuentra. Por eso es importante respetar la estructura de paquetes.

2.3 El ciclo de vida de un bean

Los beans de Spring tienen un ciclo de vida gestionado por el contenedor:

1. Instanciación. Spring crea la instancia llamando al constructor.
2. Inyección de dependencias. Spring inyecta las dependencias que el bean necesita.
3. Post-construcción. Si el bean implementa `@PostConstruct` o interfaces específicas, Spring ejecuta esos métodos.
4. Uso. El bean está listo para ser usado por otros beans.
5. Pre-destrucción. Al apagar la aplicación, Spring ejecuta los métodos anotados con `@PreDestroy`.
6. Destrucción. Spring libera el bean.

En la práctica, para un servicio normal, solo importan los pasos 1 y 2: Spring crea el servicio y le inyecta sus dependencias. El resto es transparente. Lo importante es entender que Spring controla la creación y el ciclo de vida. Tú no haces `new AlumnoService()`: Spring lo hace por ti. Y cuando otro bean necesita un `AlumnoService`, Spring le pasa la instancia ya creada.

Pregunta

¿Por qué es útil que los beans sean singletons por defecto? ¿Qué problema evita?

Respuesta razonada

Evita crear innecesariamente una instancia por uso y permite compartir colaboradores sin estado. A cambio, obliga a tratar con cuidado cualquier estado mutable compartido.

Bloque 3 - Inyección De Dependencias Por Constructor

3.1 Qué es la inyección de dependencias

La inyección de dependencias es un patrón por el cual un objeto no crea sus dependencias, sino que las recibe desde fuera. En lugar de que un servicio haga `new AlumnoRepository()`, el repositorio se le pasa como parámetro en el constructor. Eso tiene varias ventajas: Desacoplamiento. El servicio no depende de una implementación concreta, sino de una interfaz. Se puede cambiar la

- implementación sin tocar el servicio. Testabilidad. En un test, se puede pasar un mock del repositorio en lugar del real. Eso permite probar el servicio de forma
- aislada. Control. Spring gestiona la creación y el ciclo de vida de las dependencias. Tú no te preocupas por ello.
- La inyección de dependencias es uno de los pilares de Spring. Es lo que permite que las capas se conecten sin acoplarse.

3.2 Inyección por constructor vs por campo

Spring ofrece tres formas de inyectar dependencias: Por constructor. La dependencia se pasa como parámetro del constructor:

```
@Service
public class AlumnoService {
    private final AlumnoRepository repository;
    public AlumnoService(AlumnoRepository repository) {
```

```
        this.repository = repository;
    }
}
```

Por campo. La dependencia se anota con `@Autowired` en el campo:

```
@Service
public class AlumnoService {
    @Autowired
    private AlumnoRepository repository;
}
```

Por setter. La dependencia se pasa a través de un método setter anotado con `@Autowired`. De las tres, la inyección por constructor es la recomendada. Tiene varias ventajas sobre las otras: Permite campos final. El campo no puede reasignarse después de la construcción.

- Garantiza que la dependencia está disponible. No se puede crear el objeto sin pasar la dependencia.
- Facilita los tests. En un test, se pasa el mock directamente al constructor.
- Detecta dependencias circulares al arrancar. Spring falla si hay un ciclo.
- La inyección por campo, aunque es la más común en tutoriales antiguos, tiene inconvenientes: no permite final, oculta las dependencias y dificulta los tests. Por eso Spring recomienda la inyección por constructor desde hace años. Si la clase tiene un solo constructor, no hace falta anotarlo con `@Autowired`. Spring lo usa automáticamente.

3.3 Ventajas de la inyección por constructor en la práctica

Veamos un ejemplo concreto de cómo la inyección por constructor facilita los tests. Supongamos que tenemos un `AlumnoService` que depende de un `AlumnoRepository`. Con inyección por constructor, el test se ve así:

```
class AlumnoServiceTest {
    @Test
    void crear_debeGuardarAlumno() {
        AlumnoRepository repositorioMock = mock(AlumnoRepository.class);
        AlumnoService servicio = new AlumnoService(repositorioMock);

        // ... configurar el mock y probar
    }
}
```

El servicio se crea con un mock del repositorio. No hace falta arrancar Spring, ni una base de datos, ni nada. El test es rápido y aislado. Con inyección por campo, en cambio, el test tendría que usar reflexión para inyectar el mock, o arrancar Spring con `@SpringBootTest`. Eso hace los tests más lentos y más frágiles. La inyección por constructor es, por tanto, una decisión de diseño que facilita la testabilidad. Y la testabilidad es una de las mejores señales de un código bien diseñado.

Pregunta

¿Por qué la inyección por constructor facilita los tests? ¿Qué diferencia hay con la inyección por campo?

Respuesta razonada

Permite construir el servicio con mocks o fakes explícitos; las dependencias quedan visibles y obligatorias desde el momento de construcción, mientras que la inyección por campo requiere al contenedor o técnicas de reflexión.

Bloque 4 - Transformación Dto ↔ Entidad

4.1 Por qué separar DTO y entidad

En una aplicación real, los datos que viajan en JSON (DTOs) y los datos que se guardan en la base de datos (entidades) no son iguales. Una entidad puede tener campos que no deben exponerse (contraseñas, tokens, auditoría) y relaciones que no tienen sentido en la API. Un DTO puede tener campos calculados o combinados que no existen en la base de datos. Separar DTO y entidad tiene varias ventajas: Seguridad. No expones campos sensibles.

- Estabilidad. La API no cambia cada vez que cambia la tabla.
- Rendimiento. Puedes enviar solo los campos necesarios.
- Validación. Los DTOs pueden tener validaciones específicas de la API.
- Desacoplamiento. El front-end no depende de la estructura de la base de datos.
- En el Módulo 1 hemos usado solo DTOs porque no teníamos entidades. A partir del Módulo 4, cuando introduzcamos JPA, tendremos entidades y DTOs. El servicio será el encargado de transformar entre ambos.

4.2 El patrón toDTO / toEntity

La forma más común de transformar entre DTO y entidad es con dos métodos privados en el servicio: toDTO(entidad): convierte una entidad a un DTO.

- toEntity(dto): convierte un DTO a una entidad.
- Por ejemplo:

```
private AlumnoDTO toDTO(Alumno entidad) {
    AlumnoDTO dto = new AlumnoDTO();
    dto.setIdentificador(entidad.getId());
    dto.setNombre(entidad.getNombre());
    dto.setApellidos(entidad.getApellidos());
    dto.setDni(entidad.getDni());
    dto.setFechaNacimiento(entidad.getFechaNacimiento());
    dto.setCurso(entidad.getCurso());
    return dto;
}

private Alumno toEntity(AlumnoDTO dto) {
    Alumno entidad = new Alumno();
    entidad.setId(dto.getIdentificador());
    entidad.setNombre(dto.getNombre());
    entidad.setApellidos(dto.getApellidos());
    entidad.setDni(dto.getDni());
    entidad.setFechaNacimiento(dto.getFechaNacimiento());
    entidad.setCurso(dto.getCurso());
    return entidad;
}
```

Estos métodos van en el servicio, porque es el único que conoce ambos modelos. El controlador solo ve DTOs; el repositorio solo ve entidades. Cuando la transformación es muy compleja, se puede usar una librería como MapStruct, que genera el código automáticamente. Pero para empezar, los métodos manuales son suficientes y más claros.

4.3 Dónde se transforma

La transformación se hace en el servicio. No en el controlador (que no debe conocer entidades) ni en el repositorio (que no debe conocer DTOs). El flujo completo es:

Cliente → JSON → Controlador → DTO → Servicio → Entidad → Repositorio → Base de datos
Base de datos → Repositorio → Entidad → Servicio → DTO → Controlador → JSON → Cliente

El controlador recibe un DTO del cliente. El servicio lo transforma a entidad, lo guarda, y cuando lo recupera, lo transforma de nuevo a DTO. El controlador devuelve el DTO al cliente. En este punto, como todavía no tenemos entidades (las veremos en el Módulo 4), simularemos la transformación con una clase simple. Pero el patrón es el mismo: el servicio es el encargado de la transformación.

Pregunta

¿Por qué el controlador no debe conocer entidades? ¿Qué problema habría si las conociera?

Respuesta razonada

Porque acoplaría el contrato HTTP al modelo de persistencia. Los DTO permiten que API y almacenamiento evolucionen con mayor independencia.

Bloque 5 - Excepciones De Negocio

5.1 Qué son y por qué se lanzan

Una excepción de negocio es una excepción que se lanza cuando se viola una regla de negocio. Por ejemplo: Intentar crear un alumno con un DNI que ya existe.

- Intentar consultar un alumno que no existe.
- Intentar actualizar un alumno con datos inválidos.
- Intentar eliminar un recurso que tiene dependencias.
- Estas situaciones no son errores técnicos (como un fallo de red o una base de datos caída). Son situaciones que el sistema debe manejar de forma controlada. Por eso se representan con excepciones específicas, no con Exception genérica. Lanzar excepciones de negocio tiene varias ventajas: Claridad. El código expresa qué ha fallado.
- Manejo centralizado. Se pueden capturar en un manejador global.
- Respuestas HTTP correctas. Cada tipo de excepción se traduce a un código de estado.
- En una API REST, las excepciones de negocio se traducen a códigos 4xx (400, 404, 409). Las excepciones técnicas se traducen a 500.

5.2 Crear una excepción de negocio

Para crear una excepción de negocio, se extiende RuntimeException (no Exception, para no obligar a capturarla en cada método):

```
public class NegocioException extends RuntimeException {  
    public NegocioException(String mensaje) {  
        super(mensaje);  
    }  
    public NegocioException(String mensaje, Throwable causa) {  
        super(mensaje, causa);  
    }  
}
```

Esta excepción se lanza desde el servicio cuando se viola una regla:

```
if (repositorio.existePorDni(dto.getDni())) {  
    throw new NegocioException("Ya existe un alumno con el DNI " + dto.getDni());  
}
```

El servicio no captura la excepción: la deja propagar. El controlador tampoco la captura. Es un manejador global (que veremos en el Módulo 5) quien la captura y la convierte en una respuesta HTTP. En este punto, para no dejarlo sin manejar, introduciremos un manejador simple con `@ExceptionHandler` en el propio controlador. En el Módulo 5 lo haremos global con `@RestControllerAdvice`.

5.3 Manejo con `@ExceptionHandler` (introducción)

`@ExceptionHandler` es una anotación de Spring que marca un método como manejador de una excepción concreta. Cuando se lanza esa excepción en un controlador, Spring invoca el método anotado en lugar de devolver un 500 por defecto. Por ejemplo, en el controlador:

```
@ExceptionHandler(NegocioException.class)
public ResponseEntity<Map<String, Object>> handleNegocio(NegocioException ex) {
    Map<String, Object> error = Map.of(
        "status", 409,
        "error", "Conflict",
        "message", ex.getMessage()
    );
    return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
}
```

Cuando el servicio lanza `NegocioException`, Spring captura la excepción y ejecuta este método. El método construye una respuesta 409 con un JSON que describe el error. En el Módulo 5 veremos cómo centralizar estos manejadores en una sola clase con `@RestControllerAdvice`, para no tener que repetirlos en cada controlador. Pero para este punto, con `@ExceptionHandler` en el controlador es suficiente.

Pregunta

¿Por qué es mejor lanzar excepciones de negocio que devolver null o un código de error? ¿Qué ventaja tiene el manejo centralizado?

Respuesta razonada

Una excepción específica expresa una situación semántica del dominio y puede traducirse de forma centralizada al canal apropiado. null o códigos genéricos pierden contexto, y el manejo centralizado evita repetir la misma traducción en cada endpoint.

Resumen de la teoría

Servicio: capa donde vive la lógica de negocio. No conoce HTTP ni acceso a datos.

- `@Service`: anotación que marca una clase como servicio. Es una especialización de `@Component`.
- Spring detecta los servicios al arrancar y los registra como beans singleton.
- Inyección por constructor: la forma recomendada. Permite final, facilita tests y detecta ciclos.
- Transformación DTO $\leftrightarrow$ Entidad: se hace en el servicio con métodos `toDTO` y `toEntity`.
- Excepciones de negocio: se lanzan desde el servicio cuando se viola una regla.
- `@ExceptionHandler`: captura excepciones en el controlador y las convierte en respuestas HTTP.

Punto 2.3 - Capas de la aplicación: Repositorio

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar la responsabilidad de un repositorio y por qué conviene aislar el acceso a datos.

2. Diferenciar @Repository, @Service y @Component.
 3. Entender el repositorio como contrato de acceso a datos aunque, en esta etapa, lo implementemos con una clase concreta.
 4. Implementar un repositorio en memoria con ConcurrentHashMap.
 5. Usar Optional<T> y colecciones sin exponer el almacenamiento interno.
 6. Refactorizar un servicio para que delegue el almacenamiento en un repositorio.
 7. Explicar dónde deben vivir las reglas de negocio y dónde no.
 8. Razonar sobre mutabilidad, copias defensivas y concurrencia en un repositorio singleton.
 9. Preparar la arquitectura para sustituir más adelante el almacenamiento en memoria por Spring Data JPA sin cambiar el controlador.
-

Bloque 1 - Qué es un repositorio y cuál es su responsabilidad

1.1 El repositorio es la frontera con los datos

Un repositorio es la capa encargada de **guardar, recuperar, buscar y eliminar datos**. Su función no es decidir si una operación está permitida por el negocio ni construir una respuesta HTTP. Su función es ocultar al resto de la aplicación **cómo y dónde** se almacenan los datos.

Hasta 2.2, AlumnoService tenía dos responsabilidades mezcladas:

```

aplicar reglas de negocio
+
almacenar y buscar alumnos en una colección

```

En 2.3 separamos ambas responsabilidades:

```

HTTP
|
v
Controller
|
v
Service
|
v
Repository
|
v
almacenamiento

```

El controlador seguirá hablando con el servicio. El servicio seguirá aplicando las reglas. La novedad es que el servicio **ya no sabrá** si los alumnos están en una lista, en un mapa, en PostgreSQL, en MongoDB o en un servicio remoto.

Por ejemplo, el servicio no debería hacer esto:

```

return alumnos.stream()
    .filter(a -> a.getIdificador().equals(id))
    .findFirst();

```

En su lugar delegará:

```

return repositorio.buscarPorId(id);

```

La diferencia no parece enorme cuando el almacenamiento está en memoria, pero arquitectónicamente es decisiva: el servicio deja de conocer el mecanismo de persistencia.

1.2 Abstracción del almacenamiento

La fuente original introduce la idea mediante una biblioteca: el servicio actúa como el bibliotecario que decide qué operación tiene sentido y el repositorio como el archivero que sabe dónde está físicamente cada elemento. La idea importante no es la analogía, sino la **abstracción**.

Un servicio debería poder formular peticiones como:

```
buscar el alumno 17
listar alumnos
buscar por DNI
almacenar un alumno
eliminar el alumno 17
```

sin expresar cómo se realizan.

Hoy esas operaciones se resolverán contra memoria. Más adelante podrán resolverse contra una base de datos. Si mantenemos estable el contrato entre servicio y repositorio, las capas superiores necesitarán pocos o ningún cambio.

1.3 Lo que el repositorio no debe hacer

Un repositorio no debe convertirse en un segundo servicio. Evitaremos especialmente estos anti-patrones:

- **Reglas de negocio.** Decidir si un DNI duplicado está permitido pertenece al servicio.
- **Semántica HTTP.** `ResponseEntity`, `HttpStatus`, `@RequestParam`, `@PathVariable` o cabeceras no tienen lugar aquí.
- **Presentación.** No debe construir mensajes pensados para el usuario ni decidir cómo se serializa una respuesta.
- **Acoplamiento a una API concreta.** Cuando exista una entidad de persistencia separada, el repositorio trabajará con el modelo de datos, no con decisiones propias del contrato HTTP.
- **Ocultar errores devolviendo null.** Los fallos inesperados no deben convertirse silenciosamente en ausencia de datos.

En este módulo todavía utilizamos `AlumnoDTO` como objeto almacenado porque aún no hemos introducido entidades JPA. Es una simplificación pedagógica temporal, no una afirmación de que DTO y entidad sean siempre lo mismo.

Pregunta

¿Por qué sería un problema que `AlumnoRepository` devolviera directamente un `ResponseEntity<AlumnoDTO>`?

Respuesta razonada

Porque el repositorio quedaría acoplado a HTTP. Ya no sería una pieza reutilizable desde un test, una tarea batch o cualquier otro canal. La existencia o no de un dato es una cuestión de acceso a datos; convertir esa ausencia en 404 Not Found es una decisión de la capa web.

Bloque 2 - @Repository y los estereotipos de Spring

2.1 @Repository frente a @Service y @Component

Spring proporciona varios estereotipos para registrar componentes en el contexto:

```
@Component
├── @Service
├── @Repository
└── @Controller / @RestController
```

Todos permiten que Spring detecte y gestione la clase como bean, pero comunican responsabilidades distintas.

- @Component: componente genérico.
- @Service: lógica de aplicación o negocio.
- @Repository: acceso a datos.
- @Controller / @RestController: frontera web.

Ejemplo:

```
@Repository
public class AlumnoRepository {
    // acceso a datos
}
```

Usar @Component también registraría la clase, pero perderíamos información arquitectónica. @Repository expresa la intención y, cuando trabajemos con tecnologías de persistencia, participa además en la traducción de excepciones de acceso a datos hacia la jerarquía de Spring.

2.2 Detección automática

La aplicación principal está situada en el paquete raíz:

```
es.mecd.demo.miproyecto
```

Al usar @SpringBootApplication, Spring escanea ese paquete y sus subpaquetes. Por eso esta ubicación es natural:

```
es.mecd.demo.miproyecto.repository.AlumnoRepository
```

Durante el arranque, de forma simplificada, ocurre lo siguiente:

1. Spring descubre la clase anotada.
2. Construye su instancia.
3. Resuelve las dependencias de su constructor.
4. Registra el bean en el contexto.
5. Lo puede inyectar en AlumnoService.

El servicio no escribirá:

```
new AlumnoRepository();
```

sino que declarará su dependencia:

```
private final AlumnoRepository repositorio;

public AlumnoService(AlumnoRepository repositorio) {
    this.repositorio = repositorio;
}
```

2.3 ¿Interfaz o clase?

En Spring Data JPA veremos repositorios declarados normalmente como interfaces que extienden contratos proporcionados por Spring. En este punto todavía no estamos usando JPA, así que implementaremos el repositorio como **clase concreta en memoria**.

Aun así, diseñaremos su API pública como si fuese un contrato estable:

```
buscarPorId(...)
listarTodos()
buscarPorDni(...)
existePorDni(...)
guardar(...)
eliminar(...)
siguienteIdentificador()
```

El servicio depende de esas operaciones y no de la estructura interna del repositorio.

Pregunta

¿Por qué es útil pensar en una interfaz pública clara incluso cuando todavía usamos una clase concreta?

Respuesta razonada

Porque obliga a separar el **qué** del **cómo**. El servicio necesita saber qué operaciones puede pedir, pero no cómo están implementadas. Esa disciplina reduce el coste de sustituir la implementación más adelante.

Bloque 3 - Diseño de un repositorio en memoria

3.1 Elegir la estructura de datos

Para simular un almacenamiento indexado por identificador utilizaremos un mapa:

```
private final Map<String, AlumnoDTO> almacen =  
    new ConcurrentHashMap<>();
```

La clave es el identificador y el valor es el objeto almacenado.

Comparación conceptual:

```
List<AlumnoDTO>  
    buscar por ID -> recorrer elementos
```

```
Map<String, AlumnoDTO>  
    buscar por ID -> acceder por clave
```

En una base de datos, la clave primaria suele disponer de un índice. Un mapa representa mejor esa idea que una lista secuencial.

3.2 ¿Por qué ConcurrentHashMap?

Los repositorios de Spring son normalmente beans singleton. Varias peticiones pueden terminar accediendo al mismo repositorio desde distintos hilos. Un HashMap ordinario no está diseñado para modificaciones concurrentes seguras.

ConcurrentHashMap permite accesos concurrentes con garantías mucho mejores para este laboratorio:

```
private final Map<String, AlumnoDTO> almacen =  
    new ConcurrentHashMap<>();
```

Esto **no convierte el repositorio en equivalente a una base de datos**. Una base de datos aporta transacciones, restricciones, aislamiento, persistencia y muchas otras garantías. Aquí únicamente evitamos utilizar una colección evidentemente inadecuada para un bean compartido.

3.3 Operaciones del repositorio

Un repositorio básico necesita operaciones de acceso a datos bien nombradas.

Consulta por ID:

```
public Optional<AlumnoDTO> buscarPorId(String id) {  
    return Optional.ofNullable(almacen.get(id));  
}
```

Listado:

```
public List<AlumnoDTO> listarTodos() {
    return almacen.values().stream()
        .map(this::copiar)
        .sorted(Comparator.comparingInt(this::idNumerico))
        .toList();
}
```

Búsqueda por DNI:

```
public Optional<AlumnoDTO> buscarPorDni(String dni) {
    return almacen.values().stream()
        .filter(a -> Objects.equals(a.getDni(), dni))
        .findFirst()
        .map(this::copiar);
}
```

Existencia:

```
public boolean existePorDni(String dni) {
    return almacen.values().stream()
        .anyMatch(a -> Objects.equals(a.getDni(), dni));
}
```

Guardar:

```
public AlumnoDTO guardar(AlumnoDTO alumno) {
    AlumnoDTO copia = copiar(alumno);
    almacen.put(copia.getIdentificador(), copia);
    return copiar(copia);
}
```

Eliminar:

```
public boolean eliminar(String id) {
    return almacen.remove(id) != null;
}
```

3.4 Optional<T> en resultados únicos

Si un alumno puede no existir, una firma como ésta hace explícita esa posibilidad:

```
Optional<AlumnoDTO> buscarPorId(String id)
```

El llamador está obligado a considerar dos estados:

hay valor
no hay valor

Eso es preferible a un contrato ambiguo basado en null.

Para resultados múltiples usaremos listas. Una ausencia de resultados se representa con una lista vacía, no con null.

3.5 No exponer el almacenamiento interno

La fuente original ya insiste en devolver una copia de la colección y no el mapa interno. En la edición nueva hacemos la protección un poco más fuerte: **también copiamos cada DTO**.

¿Por qué? Porque esto:

```
new ArrayList<>(almacen.values())
```

crea una colección nueva, pero los elementos siguen siendo las mismas referencias mutables. Un llamador podría hacer:

```
var lista = repositorio.listarTodos();  
lista.get(0).setNombre("MODIFICADO DESDE FUERA");
```

y alterar indirectamente el objeto guardado.

La copia defensiva evita esa fuga de mutabilidad.

Pregunta

¿Por qué no basta siempre con copiar únicamente la lista exterior?

Respuesta razonada

Porque una copia superficial de la colección sigue conteniendo referencias a los mismos objetos mutables. Si esos objetos se modifican, el estado interno puede cambiar aunque el mapa o la lista originales no se hayan expuesto directamente.

Bloque 4 - El servicio entre controlador y repositorio

4.1 El servicio deja de almacenar datos

Hasta 2.2, AlumnoService tenía:

```
private final List<AlumnoDTO> alumnos = ...;
```

Después del refactor tendrá:

```
private final AlumnoRepository repositorio;  
  
public AlumnoService(AlumnoRepository repositorio) {  
    this.repositorio = repositorio;  
}
```

La consulta se reduce a:

```
public Optional<AlumnoDTO> consultar(String id) {  
    return repositorio.buscarPorId(id);  
}
```

La capa de servicio conserva lo que sí le pertenece: filtros de aplicación, reglas de negocio, promoción de curso, composición de operaciones y decisiones sobre qué hacer cuando los datos cumplen o incumplen una regla.

4.2 Regla de DNI duplicado

La fuente pregunta expresamente por qué la validación de DNI duplicado permanece en el servicio. La respuesta es arquitectónica.

El repositorio sabe responder:

```
repositorio.existePorDni(dni)
```

pero la regla:

no permitimos crear otro alumno cuando el DNI ya existe

pertenece al negocio y se expresa en AlumnoService:

```
if (repositorio.existePorDni(dto.getDni())) {  
    throw new NegocioException(  
        "Ya existe un alumno con el DNI " + dto.getDni());  
}
```

El repositorio aporta el dato; el servicio interpreta ese dato según una regla.

4.3 Generación de identificadores: actualización respecto a la fuente

La fuente original usa:

```
repositorio.contar() + 1
```

para obtener un nuevo identificador. Esa técnica es didácticamente sencilla, pero en nuestro curso acumulativo ya sabemos que puede colisionar tras borrar elementos.

Ejemplo:

```
IDs existentes: 1, 2, 3
borramos 2
contar() = 2
contar() + 1 = 3 <- colisión
```

Por eso la edición 2026 conserva la idea de delegar el almacenamiento, pero utiliza una secuencia monotónica dentro del repositorio:

```
private final AtomicInteger secuencia = new AtomicInteger(0);

public String siguienteIdentificador() {
    return String.valueOf(secuencia.incrementAndGet());
}
```

El servicio solicita el identificador cuando el DTO todavía no trae uno:

```
if (dto.getIdentificador() == null || dto.getIdentificador().isBlank()) {
    dto.setIdentificador(repositorio.siguienteIdentificador());
}
```

Así evitamos reintroducir un defecto que ya habíamos corregido en M1.

4.4 DTO y entidad: separación conceptual

La fuente introduce aquí de nuevo el patrón DTO ↔ entidad. En este módulo todavía no existe una clase JPA Alumno, así que el objeto almacenado sigue siendo AlumnoDTO.

El flujo conceptual que debemos retener es:

```
Controller
|
| DTO
v
Service
|
| modelo de almacenamiento
v
Repository
```

Cuando en M4 aparezcan entidades JPA, será el servicio —o un mapper dedicado— quien mantenga la frontera entre la representación externa y la persistente.

No crearemos una entidad ficticia sólo para aparentar una separación que aún no hemos enseñado.

4.5 Flujo completo

Una consulta queda así:

```
GET /api/v1/alumnos/1
|
v
AlumnoController.consultar("1")
|
v
```

```

AlumnoService.consultar("1")
    |
    v
AlumnoRepository.buscarPorId("1")
    |
    v
ConcurrentHashMap

```

La respuesta vuelve en dirección contraria hasta que el controlador decide entre 200 y 404.

Pregunta

¿Qué capa tendría que cambiar principalmente si sustituyéramos el mapa en memoria por una base de datos?

Respuesta razonada

La capa de repositorio. El servicio podría necesitar pequeños ajustes cuando el modelo persistente pase a ser distinto del DTO, pero el controlador no debería enterarse de cómo se almacenan los datos.

Bloque 5 - Buenas prácticas, inmutabilidad y concurrencia

5.1 Buenas prácticas de diseño

Un repositorio en memoria debe seguir reglas similares a las que exigiremos a uno real:

- nombres descriptivos: `buscarPorId`, `buscarPorDni`, `listarTodos`, `guardar`, `eliminar`;
- `Optional<T>` para resultados únicos que pueden no existir;
- listas vacías para resultados múltiples sin coincidencias;
- no devolver `null` como contrato normal;
- no exponer directamente mapas o colecciones internas;
- evitar que referencias mutables externas puedan modificar el estado sin pasar por guardar;
- no contener decisiones de negocio;
- no conocer HTTP;
- utilizar una estructura adecuada para accesos concurrentes.

5.2 Inmutabilidad observable

Nuestro repositorio no es verdaderamente inmutable: permite guardar y borrar. Lo que queremos proteger es el **encapsulamiento del estado interno**.

Esta prueba debe fallar en modificar el repositorio indirectamente:

```
List<AlumnoDTO> copia = repositorio.listarTodos();
copia.clear();
```

```
assertFalse(repositorio.listarTodos().isEmpty());
```

Y esta segunda prueba protege también los elementos:

```
AlumnoDTO externo = repositorio.buscarPorId("1").orElseThrow();
externo.setNombre("CAMBIO EXTERNO");
```

```
AlumnoDTO almacenado = repositorio.buscarPorId("1").orElseThrow();
assertNotEquals("CAMBIO EXTERNO", almacenado.getNombre());
```

La idea es sencilla: la única forma deliberada de modificar el estado almacenado debe pasar por las operaciones del repositorio.

5.3 Concurrency

La fuente propone comprobar accesos concurrentes. En lugar de limitarnos a afirmar que `ConcurrentHashMap` es seguro, conviene observar un escenario reproducible.

A nivel de HTTP podemos lanzar varias creaciones en paralelo. A nivel de test podemos usar un pool de hilos y esperar a que todos terminen.

Lo que queremos comprobar no es sólo que la aplicación “no explote”, sino propiedades concretas:

- cada creación obtiene un ID diferente;
- no se pierde ninguna operación válida;
- el repositorio puede seguir listando los datos;
- no aparecen excepciones estructurales propias de una colección no segura.

5.4 Errores comunes

Los fallos típicos de este punto son:

Síntoma	Causa probable	Corrección
Spring no puede inyectar el repositorio el servicio sigue teniendo una colección propia aparece <code>ResponseEntity</code> en el repositorio	falta <code>@Repository</code> o el paquete queda fuera del escaneo refactor incompleto	revisar anotación y paquete mover almacenamiento al repositorio
buscarPorId devuelve null modificar la lista devuelta altera el repositorio modificar un DTO devuelto altera el almacenado	fuga de HTTP contrato ambiguo se expone estado interno	devolver tipos Java del dominio/datos devolver <code>Optional</code> devolver copia defensiva
IDs duplicados después de borrar resultados cambian de orden entre ejecuciones DNI duplicado se valida dentro del repositorio	copia superficial insuficiente se usa <code>contar()+1</code> ConcurrentHashMap no garantiza orden de iteración lógica de negocio mal ubicada	copiar también elementos mutables secuencia segura o estrategia equivalente ordenar al construir la lista observable mantener la regla en el servicio

5.5 Cuando sirve un repositorio en memoria

Es útil para:

- aprendizaje;
- prototipos rápidos;
- pruebas aisladas;
- experimentos en los que no necesitamos persistencia real.

No sustituye una base de datos cuando necesitamos:

- persistencia entre reinicios;
- transacciones;
- consultas complejas;
- integridad referencial;
- grandes volúmenes;
- concurrencia y consistencia con garantías de producción.

Pregunta

¿Por qué tiene valor aprender primero la separación de capas con memoria antes de introducir JPA?

Respuesta razonada

Porque permite distinguir el patrón arquitectónico de la tecnología concreta. Si primero entendemos Controller → Service → Repository, después Spring Data JPA aparece como una implementación del acceso a datos, no como magia que mezcla todas las responsabilidades.

Resumen del punto 2.3

Al terminar este punto la aplicación tendrá tres capas diferenciadas:

```
controller/
  AlumnoController
  ExpedienteController
```

```
service/
  AlumnoService
  ExpedienteService
```

```
repository/
  AlumnoRepository
  ExpedienteRepository
```

La frontera de responsabilidades queda así:

Responsabilidad	Controller	Service	Repository
rutas, métodos HTTP y cabeceras	sí	no	no
ResponseEntity y códigos HTTP	sí	no	no
reglas de negocio	no	sí	no
filtros y composición de operaciones	no	sí	no
decidir si un DNI duplicado es válido	no	sí	no
buscar por ID/DNI	no	solicita	sí
guardar y eliminar datos	no	solicita	sí
conocer la estructura de almacenamiento	no	no	sí
thread-safety del almacenamiento en memoria	no	no	sí

El cambio fundamental es que **el almacenamiento deja de contaminar al servicio**. Eso prepara la aplicación para configuración, testing y persistencia real sin romper el contrato HTTP ya construido.

Punto 2.4 - Inyección de dependencias y configuración

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar inversión de control (IoC) e inyección de dependencias (DI) sin tratarlas como “magia” de Spring.
2. Reconocer los principales estereotipos: @Component, @Service, @Repository, @Controller y @RestController.
3. Usar inyección por constructor como patrón principal.
4. Entender @Configuration y @Bean y saber cuándo son necesarios.
5. Ejecutar lógica controlada al arrancar mediante CommandLineRunner.
6. Separar datos iniciales de la implementación de los repositorios.
7. Externalizar configuración mediante application.properties y perfiles.
8. Leer propiedades con @Value entendiendo sus ventajas y sus límites.
9. Diferenciar configuración pedagógica no sensible de secretos que nunca deberían exponerse por HTTP.

Bloque 1 - Inversión de control e inyección de dependencias

1.1 El problema de construir dependencias con new

Sin un contenedor, una clase suele fabricar sus colaboradores:

```
public class AlumnoService {  
    private final AlumnoRepository repositorio = new AlumnoRepository();  
}
```

El problema es que AlumnoService conoce dos cosas distintas:

qué dependencia necesita
+
cómo construirla

Si AlumnoRepository necesitara configuración, un cliente HTTP o una conexión a datos, el servicio tendría que conocer también esos detalles.

Con inyección por constructor:

```
private final AlumnoRepository repositorio;  
  
public AlumnoService(AlumnoRepository repositorio) {  
    this.repositorio = repositorio;  
}
```

la clase expresa **qué necesita**, pero no decide **cómo se crea**.

1.2 Inversión de control

La inversión de control significa que la aplicación deja de controlar directamente la creación y conexión de muchos de sus objetos. El contenedor de Spring asume ese trabajo.

En nuestro proyecto:

Spring crea AlumnoRepository
Spring crea AlumnoService y le entrega AlumnoRepository
Spring crea AlumnoController y le entrega AlumnoService

El flujo de dependencias queda:

```
AlumnoController  
  |  
  v  
AlumnoService  
  |  
  v  
AlumnoRepository
```

pero ninguna de esas clases necesita invocar new para construir la siguiente.

1.3 Inyección por constructor

La edición 2026 usa inyección por constructor como patrón principal:

```
@Service  
public class AlumnoService {  
  
    private final AlumnoRepository repositorio;  
  
    public AlumnoService(AlumnoRepository repositorio) {
```

```
        this.repositorio = repositorio;
    }
}
```

Ventajas:

- las dependencias obligatorias son visibles;
- pueden declararse final;
- el objeto no existe en un estado parcialmente inicializado;
- los tests pueden construir la clase directamente;
- se reducen los acoplamientos ocultos;
- las dependencias circulares aparecen claramente durante el arranque.

1.4 Inyección por campo

Esto funciona:

```
@Autowired
private AlumnoRepository repositorio;
```

pero no es el patrón principal del curso. La dependencia queda oculta dentro de la clase, no puede expresarse naturalmente como final y dificulta tests simples sin contenedor.

Con un único constructor, Spring no necesita que escribamos @Autowired:

```
public AlumnoService(AlumnoRepository repositorio) {
    this.repositorio = repositorio;
}
```

Pregunta

¿Por qué la inyección por constructor ayuda a detectar un diseño con demasiadas dependencias?

Respuesta razonada

Porque todas las dependencias aparecen en la firma del constructor. Si una clase necesita ocho o diez colaboradores, el problema resulta visible y nos obliga a preguntarnos si esa clase tiene demasiadas responsabilidades.

Bloque 2 - Estereotipos y detección de componentes

2.1 Los principales estereotipos

Spring utiliza anotaciones especializadas para comunicar el rol de una clase:

Anotación	Rol habitual
@Component	componente genérico
@Service	lógica de aplicación/negocio
@Repository	acceso a datos
@Controller	controlador MVC de vistas
@RestController	controlador REST

Todos participan en la detección de componentes, pero no son intercambiables desde el punto de vista del diseño.

2.2 Component scanning

@SpringBootApplication incluye la configuración que permite escanear componentes desde el paquete de la clase principal hacia sus subpaquetes.

Con la clase principal en:

```
es.mecd.demo.miproyecto
```

Spring encuentra:

```
es.mecd.demo.miproyecto.controller  
es.mecd.demo.miproyecto.service  
es.mecd.demo.miproyecto.repository  
es.mecd.demo.miproyecto.config
```

Si colocamos una clase anotada fuera de ese árbol, Spring puede no encontrarla automáticamente.

2.3 Qué ocurre cuando falta un bean

Un error típico es:

```
No qualifying bean of type ... available
```

Antes de añadir anotaciones al azar, revisa:

1. ¿La clase debe ser un bean?
2. ¿Tiene el estereotipo correcto?
3. ¿Está dentro del árbol de escaneo?
4. ¿Existe más de una implementación candidata?
5. ¿El constructor exige una dependencia que Spring no sabe crear?

Pregunta

¿Cómo distinguirías un fallo de component scanning de un fallo dentro del constructor del bean?

Respuesta razonada

Si Spring no descubre ninguna definición, informará de que no existe un bean candidato. Si encuentra la definición pero falla al construirla, la traza mostrará un error durante la creación del bean y normalmente una causa más profunda en su constructor o sus dependencias.

Bloque 3 - @Configuration y @Bean

3.1 Cuándo no basta con estereotipos

Podemos anotar nuestras propias clases con @Service o @Repository, pero no siempre controlamos la clase que queremos registrar.

Imagina que necesitamos un objeto de una biblioteca externa:

```
DateTimeFormatter  
ObjectMapper  
HttpClient
```

No podemos modificar su código para añadir @Component.

Para esos casos usamos una clase de configuración:

```
@Configuration  
public class AplicacionConfig {  
}
```

Y métodos @Bean:

```
@Bean
public DateTimeFormatter formatoFecha() {
    return DateTimeFormatter.ISO_LOCAL_DATE;
}
```

Spring registra el objeto devuelto y podrá inyectarlo en otros beans.

3.2 @Bean frente a @Component

La diferencia conceptual es:

@Component -> la propia clase se declara componente

@Bean -> un método de configuración crea y registra un objeto

@Bean es especialmente útil cuando:

- la clase pertenece a una biblioteca externa;
- su construcción necesita parámetros o configuración especial;
- queremos elegir explícitamente una implementación;
- queremos centralizar la construcción de un colaborador.

3.3 ObjectMapper y Spring Boot

La fuente original utiliza ObjectMapper como ejemplo de @Bean. Es importante conservar el ejemplo, pero también actualizar su interpretación.

Spring Boot ya auto-configura un ObjectMapper con integración para el ecosistema Spring y Java moderno. Por tanto, crear uno desde cero sólo para soportar LocalDate normalmente **no es necesario** y puede sustituir parte de la auto-configuración.

El ejemplo conceptual de la fuente es válido para entender @Bean:

```
@Bean
public ObjectMapper objectMapper() {
    return new ObjectMapper()
        .registerModule(new JavaTimeModule());
}
```

Pero en la práctica 2026 preferiremos personalizar el builder que usa Boot:

```
@Bean
public Jackson2ObjectMapperBuilderCustomizer jacksonCustomizer() {
    return builder -> builder.featuresToDisable(
        SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
}
```

Así enseñamos @Bean sin reemplazar innecesariamente toda la auto-configuración de Jackson.

Pregunta

¿Por qué una clase externa no puede resolverse simplemente añadiéndole @Component?

Respuesta razonada

Porque no controlamos su código fuente. @Bean permite crear y registrar una instancia desde nuestra propia configuración.

Bloque 4 - CommandLineRunner y lógica de arranque

4.1 Qué es CommandLineRunner

CommandLineRunner permite ejecutar código **después de que el contexto de Spring se haya creado** y los beans necesarios estén disponibles.

Ejemplo:

```
@Bean
CommandLineRunner cargarDatos(AlumnoRepository alumnos) {
    return args -> {
        // inicialización controlada
    };
}
```

Es útil para:

- cargar datos de demostración;
- ejecutar comprobaciones de arranque;
- inicializar estructuras pedagógicas;
- lanzar tareas puntuales al iniciar una aplicación.

No es un sustituto de herramientas de migración de base de datos ni un lugar donde esconder procesos complejos de producción.

4.2 Por qué sacar los datos del constructor del repositorio

En 2.3 los repositorios nacen con datos en sus constructores. Esto fue útil para introducir la capa sin añadir más conceptos, pero mezcla dos responsabilidades:

Repositorio -> almacenar datos

Repositorio -> decidir datos iniciales

En 2.4 moveremos la inicialización a una clase de configuración:

```
DatosInicialesConfig
|
+--> AlumnoRepository
+--> ExpedienteRepository
```

El repositorio quedará vacío al construirse y no sabrá si los datos vienen de un runner, un test o una futura base de datos.

4.3 Dependencias como parámetros del bean

Spring puede resolver los parámetros de un método @Bean:

```
@Bean
CommandLineRunner cargarDatos(
    AlumnoRepository alumnos,
    ExpedienteRepository expedientes) {
    return args -> {
        // usar ambos beans
    };
}
```

No hacemos:

```
new AlumnoRepository()
```

porque crearíamos otra instancia fuera del contenedor y romperíamos la identidad del bean gestionado por Spring.

Pregunta

¿Por qué CommandLineRunner puede recibir repositorios ya contruidos?

Respuesta razonada

Porque Spring resuelve sus dependencias al crear el bean. Cuando el runner se ejecuta, el contexto ya contiene los repositorios necesarios.

Bloque 5 - Configuración externalizada y perfiles

5.1 Por qué sacar valores del código

Los valores que cambian entre entornos no deberían obligarnos a recompilar la aplicación.

En lugar de escribir:

```
private final String nombre = "Mi Proyecto";
```

podemos definir:

```
app.nombre=Mi Proyecto Spring Boot
app.version=2.4
app.entorno=base
```

La configuración externalizada permite variar comportamiento operativo sin modificar código fuente.

5.2 @Value

La fuente introduce @Value, que sigue siendo útil para ejemplos pequeños:

```
@Value("${app.nombre}")
private String nombre;
```

También admite un valor por defecto:

```
@Value("${app.descripcion:Aplicación sin descripción}")
private String descripcion;
```

En aplicaciones con muchos parámetros relacionados, una clase tipada con @ConfigurationProperties suele escalar mejor. No la introducimos todavía como sustitución obligatoria porque el objetivo de la fuente en este punto es comprender la resolución de propiedades.

5.3 No exponer secretos

En la práctica construiremos un endpoint de diagnóstico con propiedades **deliberadamente no sensibles**:

```
app.nombre
app.version
app.entorno
```

Nunca deberíamos exponer de la misma forma:

```
contraseñas
API keys
tokens
connection strings con credenciales
```

El hecho de que una propiedad pueda inyectarse no significa que deba devolverse a un cliente.

5.4 Perfiles

Podemos crear configuraciones específicas:

```
application-dev.properties  
application-prod.properties
```

Ejemplo de desarrollo:

```
app.entorno=desarrollo  
logging.level.es.mecd.demo.miproyecto=DEBUG  
server.port=8080
```

Ejemplo de producción:

```
app.entorno=produccion  
logging.level.es.mecd.demo.miproyecto=INFO  
server.port=8080
```

Las propiedades específicas sobrescriben las de la configuración base cuando activamos el perfil.

Activación por línea de comandos:

```
./mvnw spring-boot:run \  
-Dspring-boot.run.profiles=dev
```

O con el JAR:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar \  
--spring.profiles.active=prod
```

5.5 Orden de precedencia

Spring Boot dispone de varias fuentes de configuración. Para este curso basta comprender una idea: un valor más específico del entorno puede sobrescribir el valor base sin cambiar el código.

No conviene memorizar una lista enorme de precedencias en esta fase; sí conviene aprender a inspeccionar qué perfil está activo y qué valor efectivo recibe la aplicación.

Pregunta

¿Qué problema evitamos al mantener perfiles separados en lugar de editar manualmente application.properties antes de cada despliegue?

Respuesta razonada

Evitamos convertir el despliegue en una edición manual propensa a errores y podemos versionar configuraciones no sensibles específicas de cada entorno de forma explícita y reproducible.

Resumen del punto 2.4

Después de este punto podremos leer la arquitectura como un grafo de dependencias gestionado por Spring:

```
Controller  
|  
Service  
|  
Repository
```

Spring crea y conecta los beans
Configuration crea beans adicionales

CommandLineRunner inicializa datos pedagógicos
application*.properties externaliza valores
profiles seleccionan configuración por entorno

Las ideas fundamentales son:

- IoC: el contenedor controla la creación y conexión de beans;
 - DI: cada clase recibe sus dependencias;
 - constructor injection: patrón principal del curso;
 - estereotipos: expresan intención arquitectónica;
 - @Configuration + @Bean: registran objetos que no se detectan por estereotipos;
 - CommandLineRunner: ejecuta inicialización tras construir el contexto;
 - configuración externalizada: evita codificar valores operativos;
 - perfiles: permiten variar configuración entre entornos sin tocar el código.
-

Profundización: ciclo de vida, formas de inyección y orden de arranque

Las tres formas de inyección

La fuente distingue explícitamente tres mecanismos y conviene verlos juntos.

Constructor — recomendado:

```
private final AlumnoRepository repositorio;

public AlumnoService(AlumnoRepository repositorio) {
    this.repositorio = repositorio;
}
```

Ventajas: dependencia obligatoria, campo final, construcción completa y tests sencillos.

Campo — frecuente en tutoriales antiguos:

```
@Autowired
private AlumnoRepository repositorio;
```

Funciona, pero oculta dependencias y obliga a que la inyección ocurra después de construir el objeto.

Setter — útil sólo cuando la dependencia es deliberadamente opcional o reconfigurable:

```
private Auditor auditor;

@Autowired
public void setAuditor(Auditor auditor) {
    this.auditor = auditor;
}
```

No utilizaremos setter injection por defecto. La regla del curso sigue siendo: constructor salvo que exista una razón explícita para otra cosa.

Ciclo de vida de un bean

Un bean no aparece mágicamente “ya hecho”. El contenedor sigue un ciclo de vida. Simplificando:

1. descubrir definición
2. instanciar
3. resolver e inyectar dependencias
4. ejecutar callbacks de inicialización
5. dejar el bean disponible
6. ejecutar callbacks de destrucción al cerrar el contexto

Spring soporta callbacks como @PostConstruct y @PreDestroy:

```

@PostConstruct
void inicializar() {
    // trabajo ligero posterior a la inyección
}

@PreDestroy
void cerrar() {
    // liberación controlada si fuera necesaria
}

```

No usaremos estas anotaciones para esconder carga de datos que ya hemos decidido expresar mediante `CommandLineRunner`, pero es importante conocer dónde encajan.

Múltiples implementaciones

Si existen varios beans compatibles con el mismo tipo, Spring necesita saber cuál inyectar. En módulos posteriores podremos utilizar `@Primary` o `@Qualifier` cuando exista una necesidad real. El punto importante aquí es que IoC no elimina las decisiones de diseño: sólo centraliza y hace explícita la construcción del grafo de objetos.

Varios `CommandLineRunner`

La fuente muestra que puede haber más de un runner. Si el orden importa, se puede expresar:

```

@Bean
@Order(1)
CommandLineRunner cargarAlumnos(AlumnoRepository repo) {
    return args -> { /* ... */ };
}

@Bean
@Order(2)
CommandLineRunner cargarExpedientes(ExpedienteRepository repo) {
    return args -> { /* ... */ };
}

```

Sin una restricción de orden, no debemos escribir código que dependa accidentalmente de cuál runner fue descubierto primero.

Más ejemplos de `application.properties`

Además de nuestras propiedades `app.*`, Spring Boot dispone de configuración propia:

```

server.port=8080
spring.application.name=mi-proyecto
logging.level.es.mecd.demo.miproyecto=DEBUG

```

`server.port` cambia el puerto HTTP. `spring.application.name` da un nombre lógico a la aplicación. `logging.level.<paquete>` ajusta el nivel de log.

Estas propiedades sólo se vuelven efectivas al arrancar el contexto con esa configuración. En un proyecto normal, cambiar una propiedad requiere reiniciar la aplicación salvo que exista infraestructura específica para recarga.

Perfil de test

La fuente enumera también `application-test.properties`. Aunque todavía no necesitaremos una configuración compleja de test, conviene reconocer el patrón completo:

```

application.properties
application-dev.properties

```

```
application-test.properties
application-prod.properties
```

Esto nos permitirá, cuando aparezcan bases de datos, usar recursos distintos por entorno sin cambiar el código Java.

Singleton no significa “global mutable sin cuidado”

Que un bean tenga alcance singleton significa que el contenedor mantiene una instancia por contexto, no que debemos usarla como almacén arbitrario de estado de petición. Un servicio singleton puede ser perfectamente seguro si sus dependencias y métodos no mantienen estado temporal compartido.

Ejemplo peligroso:

```
@Service
public class InformeService {
    private String usuarioActual;

    public void generar(String usuario) {
        this.usuarioActual = usuario;
        // dos peticiones podrían pisarse
    }
}
```

Ejemplo preferible:

```
public Informe generar(String usuario) {
    // usuario es estado local de esta invocación
}
```

La misma reflexión que hicimos con controladores se aplica a servicios, repositorios y configuraciones.

Nombre de los beans

Por defecto, una clase `AlumnoService` registrada por estereotipo recibe un nombre de bean equivalente a `alumnoService`. En métodos `@Bean`, el nombre por defecto es el nombre del método:

```
@Bean
DateTimeFormatter formatoFecha() { ... }
```

produce un bean llamado `formatoFecha`.

También puede nombrarse explícitamente:

```
@Bean("formatoFechaCorto")
DateTimeFormatter formatoFecha() { ... }
```

No necesitaremos normalmente referirnos al nombre porque la inyección se resuelve por tipo, pero conocerlo ayuda a entender diagnósticos y escenarios con múltiples beans.

@Component frente a @Bean: ejemplo completo

Para una clase propia:

```
@Service
public class NotificacionService {
}
```

es más directo que:

```
@Configuration
class ServiciosConfig {
    @Bean
    NotificacionService notificacionService() {
```

```
        return new NotificacionService();
    }
}
```

En cambio, para una clase externa o una construcción especial, `@Bean` expresa mejor la decisión:

```
@Bean
Clock reloj() {
    return Clock.systemUTC();
}
```

El resto de la aplicación puede inyectar `Clock` sin saber cómo se creó. Este patrón será especialmente útil en tests y configuración avanzada.

Qué significa que `@Configuration` sea especial

Una clase anotada con `@Configuration` no es simplemente un contenedor de métodos estáticos. Spring la registra como parte de la configuración del contexto y procesa sus métodos `@Bean`. El resultado de cada método se convierte en una definición gestionada por el contenedor.

Esto permite expresar dependencias entre beans sin asumir que cada llamada Java crea un objeto independiente. A nivel pedagógico, la idea que debemos conservar es:

método `@Bean` -> definición gestionada por Spring
no -> fábrica manual invocada libremente por el resto del código

Por eso las clases de negocio no deberían llamar directamente a métodos de configuración para obtener colaboradores. Deben declararlos como dependencias y dejar que Spring los resuelva.

Configuración base y sobrescritura por entorno

Imagina esta base:

```
app.nombre=Curso Spring Boot
app.entorno=base
server.port=8080
```

Y desarrollo:

```
app.entorno=desarrollo
logging.level.es.mecd.demo.miproyecto=DEBUG
```

El perfil no necesita repetir `app.nombre` ni `server.port`: hereda la base y sobrescribe únicamente lo específico. Esa composición evita duplicación y reduce el riesgo de que configuraciones paralelas diverjan innecesariamente.

Diagnóstico del grafo de dependencias

Cuando Spring no puede construir el contexto, la excepción suele mostrar una cadena de dependencias. Léela desde el bean que falló hacia la causa raíz. Por ejemplo, si `AlumnoController` depende de `AlumnoService`, que depende de `AlumnoRepository`, un repositorio no registrado puede aparecer como causa final aunque el mensaje superior mencione el controlador. La habilidad útil no es memorizar la excepción, sino reconstruir el grafo de dependencias y localizar el primer bean que Spring no pudo resolver.

Esta lectura será especialmente importante cuando el proyecto tenga más configuraciones y tests de contexto.

Una última regla práctica sobre configuración

La configuración debe cambiar comportamiento operativo, no ocultar decisiones de negocio arbitrarias. Un puerto, un nombre de aplicación o un nivel de log son buenos candidatos. En cambio, una regla esencial del dominio no debería desaparecer dentro de una propiedad sin una explicación clara.

Externalizar no significa desresponsabilizar al diseño: seguimos necesitando saber qué valores son seguros, cuáles son obligatorios, qué defaults existen y qué ocurre cuando una propiedad falta.

Punto 2.5 - Introducción al testing

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar por qué probar una aplicación por capas reduce el coste de detectar errores.
 2. Diferenciar tests unitarios, tests de integración, tests web y pruebas end-to-end.
 3. Usar JUnit 5: @Test, @BeforeEach, aserciones y assertThrows.
 4. Usar Mockito para crear dobles, configurar respuestas y verificar interacciones.
 5. Probar AlumnoService sin arrancar Spring ni usar un repositorio real.
 6. Probar un controlador con @WebMvcTest y MockMvc.
 7. Sustituir la dependencia del servicio por @MockitoBean en Spring Boot 3.5.x.
 8. Comprender por qué @MockBean aparece en la fuente histórica pero está deprecado en la línea actual del curso.
 9. Escribir tests reproducibles e independientes.
 10. Aplicar el mismo patrón a ExpedienteService.
-

Bloque 1 - Por qué testear

1.1 Un test es una especificación ejecutable

Un test automatizado no sirve únicamente para “ver si compila”. Expresa una expectativa concreta y la vuelve ejecutable.

Ejemplo:

Dado un alumno existente con ID 1
cuando consulto ese ID
entonces el servicio devuelve el alumno

Cuando el código cambia, el test vuelve a comprobar esa expectativa.

1.2 La pirámide de tests

La fuente introduce la pirámide clásica:

```

      pocos end-to-end
    -----
    integración/web
    -----
      muchos unitarios
  
```

Los tests unitarios suelen ser:

- rápidos;
- aislados;
- baratos de ejecutar;
- fáciles de localizar cuando fallan.

Las pruebas que arrancan más infraestructura validan más piezas simultáneamente, pero son más lentas y pueden dificultar localizar la causa exacta de un fallo.

La idea no es que exista una proporción matemática obligatoria, sino que **la mayor parte de las reglas pequeñas deberían poder comprobarse sin arrancar toda la aplicación.**

1.3 Qué probar en cada capa

Para nuestro proyecto:

Capa	Tipo de test principal	Qué queremos comprobar
Repository en memoria	unitario directo	buscar, guardar, borrar, copias defensivas
Service	unitario con Mockito	reglas de negocio y delegación
Controller	slice web	rutas, status, JSON, cabeceras
Aplicación completa	integración/smoke	contexto y recorrido real

Pregunta

¿Por qué no deberíamos probar una regla de DNI duplicado únicamente con una petición HTTP de extremo a extremo?

Respuesta razonada

Porque la regla pertenece al servicio y puede comprobarse directamente con un test unitario rápido. La prueba HTTP sigue siendo útil para verificar la traducción a 409, pero no debería ser la única protección de la regla.

Bloque 2 - JUnit 5

2.1 JUnit Jupiter

Spring Boot incluye JUnit 5 a través de spring-boot-starter-test.

Un test mínimo:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculadoraTest {

    @Test
    void sumarDosMasTresDevuelveCinco() {
        assertEquals(5, 2 + 3);
    }
}
```

2.2 Ciclo de vida

Anotaciones importantes:

- @Test: método de prueba.
- @BeforeEach: preparación antes de cada test.
- @AfterEach: limpieza posterior.
- @BeforeAll: preparación una vez para la clase.
- @AfterAll: limpieza final.

En nuestros tests de servicio usaremos @BeforeEach para reconstruir el mock y el objeto probado antes de cada escenario.

2.3 Aserciones

Ejemplos:

```
assertEquals(esperado, real);
assertTrue(condicion);
assertFalse(condicion);
assertNotNull(valor);
assertNull(valor);
```

Para excepciones:

```
NegocioException ex = assertThrows(
    NegocioException.class,
    () -> servicio.crear(dto));
```

Y podemos comprobar el mensaje:

```
assertTrue(ex.getMessage().contains("DNI"));
```

2.4 Tests independientes

Un test no debería depender del orden en que JUnit ejecute otros tests. Evitaremos:

test A crea dato

↓

test B asume que el dato de A sigue ahí

Cada test prepara el escenario que necesita.

Pregunta

¿Qué diferencia hay entre assertEquals y assertThrows?

Respuesta razonada

assertEquals compara valores. assertThrows expresa que el comportamiento correcto de un escenario concreto es lanzar una excepción del tipo esperado.

Bloque 3 - Mockito

3.1 Por qué usamos mocks

AlumnoService depende de AlumnoRepository. Para probar sólo la lógica del servicio no necesitamos un ConcurrentHashMap real ni Spring.

Creamos un mock:

```
AlumnoRepository repositorio = mock(AlumnoRepository.class);
AlumnoService servicio = new AlumnoService(repositorio);
```

El mock permite programar respuestas:

```
when(repositorio.buscarPorId("1"))
    .thenReturn(Optional.of(alumno));
```

3.2 Stubbing

Configurar el comportamiento se conoce habitualmente como *stubbing*.

```
when(repositorio.existePorDni("DNI-DEMO-01"))
    .thenReturn(true);
```

El test controla el escenario sin depender del estado de un repositorio real.

3.3 Verificación de interacciones

Mockito también puede comprobar que una colaboración ocurrió:

```
verify(repositorio).guardar(any(AlumnoDTO.class));
```

O que no ocurrió:

```
verify(repositorio, never()).guardar(any());
```

Esto es especialmente útil en escenarios de error. Si el DNI ya existe, no basta con comprobar que se lanza una excepción; también podemos exigir que **no se intente guardar**.

3.4 No sobreverificar

Un test frágil verifica demasiados detalles internos y falla ante cualquier refactor legítimo.

Conviene verificar interacciones cuando forman parte del comportamiento relevante:

si la regla rechaza la creación -> no guardar

si la creación es válida -> guardar una vez

No necesitamos verificar cada llamada trivial.

Pregunta

¿Qué tipo de error detecta `verify(repositorio, never()).guardar(...)` en el caso de DNI duplicado?

Respuesta razonada

Detecta una implementación que lanza la excepción demasiado tarde, después de haber modificado el almacenamiento.

Bloque 4 - Tests unitarios del servicio

4.1 Estructura Arrange-Act-Assert

Un estilo claro divide mentalmente cada prueba en tres fases:

Arrange -> preparar datos y mocks

Act -> ejecutar la operación

Assert -> comprobar resultado e interacciones

Ejemplo:

```
@Test
void consultarExistenteDevuelveAlumno() {
    AlumnoDTO alumno = alumno("1", "Ana", "DNI-DEMO-01");
    when(repositorio.buscarPorId("1"))
        .thenReturn(Optional.of(alumno));

    Optional<AlumnoDTO> resultado = servicio.consultar("1");

    assertTrue(resultado.isPresent());
    assertEquals("Ana", resultado.orElseThrow().getNombre());
}
```

4.2 Consulta no encontrada

```
@Test
void consultarInexistenteDevuelveOptionalVacio() {
    when(repositorio.buscarPorId("999"))
        .thenReturn(Optional.empty());

    Optional<AlumnoDTO> resultado = servicio.consultar("999");

    assertTrue(resultado.isEmpty());
}
```

4.3 Regla de DNI duplicado

```
@Test
void crearConDniDuplicadoLanzaNegocioException() {
    AlumnoDTO dto = alumno(null, "Otra Ana", "DNI-DEMO-01");
    when(repositorio.existePorDni("DNI-DEMO-01"))
        .thenReturn(true);

    assertThrows(NegocioException.class,
        () -> servicio.crear(dto));

    verify(repositorio, never()).guardar(any());
}
```

4.4 Creación correcta y evolución respecto a la fuente

La fuente original configura repositorio.contar() porque en aquella versión el ID se calculaba con contar()+1. Nuestro curso ya utiliza siguienteIdentificador() para evitar colisiones.

```
@Test
void crearValidoAsignaIdYGuarda() {
    AlumnoDTO dto = alumno(null, "María", "DNI-DEMO-03");

    when(repositorio.existePorDni("DNI-DEMO-03"))
        .thenReturn(false);
    when(repositorio.siguienteIdentificador())
        .thenReturn("3");
    when(repositorio.guardar(any(AlumnoDTO.class)))
        .thenAnswer(inv -> inv.getArgument(0));

    AlumnoDTO creado = servicio.crear(dto);

    assertEquals("3", creado.getIdentificador());
    verify(repositorio).guardar(any(AlumnoDTO.class));
}
```

El test refleja nuestro contrato actual, no una copia ciega del ejemplo histórico.

Pregunta

¿Por qué es mejor actualizar el test a siguienteIdentificador() que conservar contar() sólo para parecerse a la fuente?

Respuesta razonada

Porque el test debe proteger el comportamiento real que hemos decidido mantener. La fuente aporta la intención pedagógica; el curso acumulativo no debe reintroducir un defecto ya corregido.

Bloque 5 - Tests web con @WebMvcTest y MockMvc

5.1 Slice test

@WebMvcTest carga una parte limitada del contexto orientada a Spring MVC, en lugar de arrancar toda la aplicación.

```
@WebMvcTest(AlumnoController.class)
class AlumnoControllerTest {
}
```

Podemos probar:

- mappings;
- status HTTP;
- JSON;
- cabeceras;
- serialización/deserialización;
- manejadores de excepciones del controlador.

5.2 MockMvc

Spring inyecta un cliente de prueba:

```
@Autowired
private MockMvc mockMvc;
```

Y podemos simular peticiones sin abrir un puerto real:

```
mockMvc.perform(get("/api/v1/alumnos/1"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.id").value("1"));
```

5.3 @MockBean de la fuente y @MockitoBean en 2026

La fuente original usa:

```
@MockBean
private AlumnoService service;
```

En la línea actual del curso, Spring Boot 3.5.x marca @MockBean como deprecado y la alternativa moderna es @MockitoBean del soporte de *bean override* de Spring Framework.

Usaremos:

```
import org.springframework.test.context.bean.override.mockito.MockitoBean;
```

```
@MockitoBean
private AlumnoService service;
```

Pedagógicamente cumple la misma función: registrar en el contexto de test un mock que sustituye al servicio real.

5.4 GET existente y 404

```
when(service.consultar("1"))
    .thenReturn(Optional.of(alumno));

mockMvc.perform(get("/api/v1/alumnos/1"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.nombre").value("Ana"));
```

Para ausencia:

```
when(service.consultar("999"))
    .thenReturn(Optional.empty());

mockMvc.perform(get("/api/v1/alumnos/999"))
    .andExpect(status().isNotFound());
```

5.5 POST y Location

El test debe proteger el contrato de 2.1:

```
when(service.crear(any(AlumnoDTO.class)))
    .thenReturn(creadoConId3);

mockMvc.perform(post("/api/v1/alumnos")
    .contentType(MediaType.APPLICATION_JSON)
    .content(json))
    .andExpect(status().isCreated())
    .andExpect(header().string("Location", "/api/v1/alumnos/3"))
    .andExpect(jsonPath("$.id").value("3"));
```

Pregunta

¿Por qué @WebMvcTest no necesita un AlumnoRepository real?

Respuesta razonada

Porque estamos probando la capa web. El controlador recibe un mock de AlumnoService, así que la cadena se corta antes del repositorio.

Resumen del punto 2.5

La estrategia queda:

Service

- > JUnit + Mockito
- > sin Spring
- > reglas rápidas y aisladas

Controller

- > @WebMvcTest + MockMvc + @MockitoBean
- > contexto web reducido
- > mappings/status/JSON/cabeceras

Aplicación completa

- > tests de integración y CI
- > recorrido real

El objetivo no es maximizar el número de tests, sino colocar cada comprobación en el nivel más barato que pueda demostrarla con suficiente confianza.

Profundización: tipos de test, rojo-verde-refactor y buenas prácticas

Tipos de test

La fuente distingue varios niveles que no deben mezclarse.

Test unitario. Prueba una unidad pequeña, normalmente una clase, sustituyendo colaboradores externos cuando sea necesario. `AlumnoServiceTest` pertenece aquí.

Test de integración. Comprueba que varias piezas reales funcionan juntas. Puede arrancar un contexto Spring, conectar con infraestructura o verificar integración de configuración.

Test web o slice. `@WebMvcTest` se sitúa entre ambos extremos: carga la infraestructura MVC necesaria para probar la frontera web, pero no toda la aplicación.

End-to-end. Recorre la aplicación de forma muy parecida a un cliente real. Es valioso, pero más lento y con más posibles causas de fallo.

Ningún nivel sustituye completamente a los demás.

El ciclo rojo-verde-refactor

La fuente introduce el ciclo clásico de TDD:

ROJO -> escribir una prueba que falla por el comportamiento que falta
VERDE -> escribir el mínimo código necesario para cumplirla
REFACTOR -> mejorar estructura manteniendo la prueba verde

No estamos obligados a desarrollar todo el curso con TDD estricto, pero el ciclo enseña una disciplina útil: una refactorización debería conservar comportamiento observable.

Ejemplo aplicado a DNI duplicado:

1. escribimos un test que exige `NegocioException`;
2. vemos el fallo;
3. añadimos la regla al servicio;
4. el test pasa;
5. podemos reorganizar la implementación mientras siga pasando.

Buenas prácticas en tests de servicio

Un test de servicio de calidad debería:

- probar un escenario claramente nombrado;
- preparar sólo los datos necesarios;
- aislar dependencias externas;
- comprobar resultado y, cuando aporta valor, interacciones;
- no depender de otros tests;
- evitar `Thread.sleep` y temporizaciones arbitrarias;
- no repetir lógica de producción dentro del propio test;
- usar datos comprensibles, no números mágicos sin explicación.

Casos de éxito y error

Si sólo probamos el camino feliz, dejamos sin protección precisamente las reglas que suelen romperse.

Para `AlumnoService.crear` son relevantes al menos:

DNI nuevo -> crea y guarda
DNI duplicado -> excepción y no guarda
ID ausente -> solicita identificador
ID explícito -> conserva contrato decidido

Para consulta:

ID existente -> Optional con valor
ID inexistente -> Optional vacío

thenAnswer

La fuente utiliza thenAnswer para simular un repositorio que devuelve el mismo objeto que recibe:

```
when(repositorio.guardar(any(AlumnoDTO.class)))
    .thenAnswer(invocation -> invocation.getArgument(0));
```

Esto permite que el test siga centrado en el servicio sin implementar un repositorio falso completo.

Verificación con argumentos

Además de comprobar que guardar fue llamado, podemos capturar el argumento:

```
ArgumentCaptor<AlumnoDTO> captor =
    ArgumentCaptor.forClass(AlumnoDTO.class);

verify(repositorio).guardar(captor.capture());
assertEquals("3", captor.getValue().getIdificador());
```

No es necesario en todos los tests, pero resulta útil cuando queremos verificar qué objeto se envió al colaborador.

Más verificaciones de MockMvc

MockMvc no se limita al status. Puede comprobar:

```
.andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
.andExpect(jsonPath("$.nombre").value("Ana"))
.andExpect(header().string("Location", "/api/v1/alumnos/3"));
```

También podemos comprobar un cuerpo vacío:

```
.andExpect(status().isNoContent())
.andExpect(content().string(""));
```

Y un error de negocio:

```
.andExpect(status().isConflict())
.andExpect(jsonPath("$.status").value(409));
```

La capa web se considera bien probada cuando verificamos la semántica HTTP relevante, no cuando duplicamos toda la lógica del servicio.

Anotaciones y aserciones de JUnit 5 en conjunto

Un ejemplo más completo muestra el ciclo de vida:

```
class EjemploTest {

    @BeforeAll
    static void antesDeTodos() {
        // una vez para la clase
    }

    @BeforeEach
    void antesDeCadaTest() {
        // antes de cada caso
    }

    @Test
    void casoUno() {
        assertEquals(2, 1 + 1);
    }
}
```

```

@Test
void casoDos() {
    assertFalse(false);
}

@AfterEach
void despuesDeCadaTest() {
    // limpieza local
}

@AfterAll
static void despuesDeTodos() {
    // una vez al final
}
}

```

@BeforeAll y @AfterAll son estáticos con el ciclo de vida por defecto de JUnit. No debemos usarlos como almacén compartido de estado mutable que haga dependientes los tests.

Aserciones adicionales útiles:

```

assertAll(
    () -> assertEquals("1", alumno.getIdentificador()),
    () -> assertEquals("Ana", alumno.getNombre()),
    () -> assertNotNull(alumno.getFechaNacimiento())
);

```

assertAll permite evaluar varias propiedades relacionadas y reportar múltiples fallos de un mismo objeto.

Mock, stub, fake y spy

La fuente se centra correctamente en mocks, pero conviene distinguir términos habituales.

- **stub**: devuelve respuestas prefijadas;
- **mock**: además permite verificar interacciones;
- **fake**: implementación simplificada pero funcional, por ejemplo un repositorio en memoria;
- **spy**: envuelve un objeto real permitiendo observar o sustituir parte de su comportamiento.

Mockito puede crear mocks y spies, pero no debemos usar un spy si un mock expresa mejor el aislamiento que queremos.

Ejemplo de creación:

```
AlumnoRepository repositorio = mock(AlumnoRepository.class);
```

Configuración de varias respuestas:

```

when(repositorio.buscarPorId("1"))
    .thenReturn(Optional.of(ana));
when(repositorio.buscarPorId("999"))
    .thenReturn(Optional.empty());

```

Verificación del número de llamadas:

```

verify(repositorio, times(1)).buscarPorId("1");
verifyNoMoreInteractions(repositorio);

```

verifyNoMoreInteractions debe usarse con moderación: puede volver frágil un test si verificamos detalles no importantes.

Tests del repositorio en memoria

Aunque la práctica principal de la fuente se centra en servicio y controlador, su objetivo de “testear cada capa” se completa con tests directos del repositorio.

No necesitamos mocks:

```
class AlumnoRepositoryTest {

    private AlumnoRepository repositorio;

    @BeforeEach
    void setUp() {
        repositorio = new AlumnoRepository();
    }

    @Test
    void guardarYBuscarDevuelveCopiaDelAlumno() {
        AlumnoDTO alumno = new AlumnoDTO(
            "10", "Eva", "Ramos", "DNI-10",
            LocalDate.of(2011, 2, 3), "4º");

        repositorio.guardar(alumno);

        AlumnoDTO encontrado = repositorio.buscarPorId("10")
            .orElseThrow();
        assertEquals("Eva", encontrado.getNombre());
    }

    @Test
    void eliminarInexistenteDevuelveFalse() {
        assertFalse(repositorio.eliminar("999"));
    }
}
```

Si en 2.4 eliminamos la carga del constructor, estos tests deberán preparar expresamente los datos que necesiten. Eso es positivo: hace visible el escenario.

Test de copia defensiva

El repositorio debe proteger su almacenamiento:

```
@Test
void modificarDtoConsultadoNoModificaElAlmacenado() {
    AlumnoDTO original = new AlumnoDTO(
        "1", "Ana", "García", "DNI-DEMO-01",
        LocalDate.of(2010, 5, 12), "5º Primaria");
    repositorio.guardar(original);

    AlumnoDTO externo = repositorio.buscarPorId("1").orElseThrow();
    externo.setNombre("CAMBIO EXTERNO");

    assertEquals(
        "Ana",
        repositorio.buscarPorId("1").orElseThrow().getNombre());
}
```

Este test documenta una propiedad arquitectónica que sería fácil romper al simplificar el código.

Test de concurrencia determinista

No queremos un test que “a veces pasa”. Podemos coordinar varios hilos con `ExecutorService` y esperar a todos los `Future`:

```
ExecutorService pool = Executors.newFixedThreadPool(4);
try {
    List<Callable<String>> tareas = IntStream.range(0, 20)
        .mapToObj(i -> (Callable<String>) repositorio::siguienteIdentificador)
        .toList();

    List<String> ids = pool.invokeAll(tareas).stream()
        .map(f -> {
            try {
                return f.get();
            } catch (Exception e) {
                throw new RuntimeException(e);
            }
        })
        .toList();

    assertEquals(20, new HashSet<>(ids).size());
} finally {
    pool.shutdownNow();
}
```

No demuestra propiedades de una base de datos, pero sí protege la decisión concreta de nuestra secuencia concurrente.

Qué carga `@WebMvcTest`

`@WebMvcTest(AlumnoController.class)` concentra el contexto en la capa MVC. Incluye infraestructura necesaria para:

- controller;
- conversión HTTP;
- Jackson relacionado con la web;
- validación web disponible;
- `MockMvc`;
- componentes MVC relevantes.

No pretende cargar servicios y repositorios reales. Por eso sustituimos el servicio mediante `@MockitoBean`.

Test de JSON mal formado

También podemos proteger el 400 heredado:

```
mockMvc.perform(post("/api/v1/alumnos")
    .contentType(MediaType.APPLICATION_JSON)
    .content("{\"nombre\":\"Pedro\"}"))
    .andExpect(status().isBadRequest());
```

El mock del servicio ni siquiera debería ser relevante: Jackson falla antes de invocarlo.

Test de media type no soportado

```
mockMvc.perform(post("/api/v1/alumnos")
    .contentType(MediaType.TEXT_PLAIN)
    .content("hola"))
    .andExpect(status().isUnsupportedMediaType());
```

Así convertimos en regresiones automáticas los laboratorios HTTP que antes ejecutábamos manualmente.

Qué hace que un test sea mantenible

La calidad de una suite depende tanto de su diseño como de su cantidad. Un buen test debería fallar cuando se rompe el contrato que protege y permanecer verde ante refactors internos que no cambian ese contrato.

Ejemplo de test demasiado acoplado:

```
verify(repositorio, times(1)).listarTodos();
verify(repositorio, times(1)).buscarPorId("1");
verifyNoMoreInteractions(repositorio);
```

si la operación realmente sólo promete “consultar un alumno”. Cambiar una estrategia interna podría romper el test sin afectar al usuario.

Ejemplo mejor:

```
Optional<AlumnoDTO> resultado = servicio.consultar("1");
assertTrue(resultado.isPresent());
assertEquals("Ana", resultado.orElseThrow().getNombre());
```

Y verificamos una interacción sólo cuando forma parte de la regla, por ejemplo que no se guarda tras un DNI duplicado.

Datos de prueba expresivos

Los datos deben ayudar a leer el escenario. Es preferible:

```
AlumnoDTO duplicado = new AlumnoDTO(
    null, "Otra Ana", "Duplicada", "DNI-DEMO-01",
    LocalDate.of(2010, 1, 1), "5º Primaria");
```

que una secuencia de valores crípticos como:

```
new AlumnoDTO(null, "a", "b", "x", null, "c")
```

Cuando el mismo constructor aparece muchas veces podemos extraer un helper de test:

```
private AlumnoDTO alumno(String id, String nombre, String dni) {
    return new AlumnoDTO(
        id, nombre, "Apellidos", dni,
        LocalDate.of(2010, 1, 1), "5º Primaria");
}
```

El helper pertenece al test y no debe filtrarse al código de producción sólo para facilitar pruebas.

Probar excepciones sin capturarlas manualmente

Evita:

```
try {
    servicio.crear(dto);
    fail("Debería fallar");
} catch (NegocioException ex) {
    // ...
}
```

JUnit ofrece una forma más declarativa:

```
NegocioException ex = assertThrows(
    NegocioException.class,
    () -> servicio.crear(dto));
```

Esto hace más evidente cuál es el comportamiento esperado.

@SpringBootTest no es “mejor” por cargar más

Un error común es pensar que un test es más serio si arranca todo Spring. Cargar más infraestructura sólo tiene sentido cuando esa infraestructura forma parte de lo que queremos verificar.

Para una regla pura de AlumnoService, esto:

```
AlumnoRepository repo = mock(AlumnoRepository.class);
AlumnoService service = new AlumnoService(repo);
```

es más rápido y más preciso que un contexto completo.

Usaremos @SpringBootTest para pruebas donde la integración entre beans, configuración y contexto sí sea el objeto del test.

Pruebas parametrizadas

JUnit 5 soporta tests parametrizados. Aunque no son imprescindibles en este punto, conviene conocerlos porque reducen duplicación cuando el mismo contrato se aplica a varios valores.

Ejemplo conceptual:

```
@ParameterizedTest
@ValueSource(strings = {"999", "no-existe", "-1"})
void consultarIdsInexistentesDevuelveVacio(String id) {
    when(repositorio.buscarPorId(id)).thenReturn(Optional.empty());
    assertTrue(servicio.consultar(id).isEmpty());
}
```

No debemos parametrizar por moda; sólo cuando varios casos comparten realmente la misma expectativa.

Organización de nombres de tests

Existen estilos diferentes:

```
metodoEscenarioResultado
shouldResultadoWhenEscenario
Given_When_Then
```

El curso no exige una única gramática, pero sí claridad y consistencia. Este proyecto utilizará preferentemente nombres españoles descriptivos como:

```
crearConDniDuplicadoLanzaNegocioException()
consultarInexistenteDevuelveOptionalVacio()
crearDevuelve201LocationYJson()
```

El valor de ejecutar la suite completa

Un test aislado puede pasar mientras una regresión en otra capa queda oculta. Por eso después de desarrollar un caso concreto ejecutamos:

```
./mvnw test
```

La suite completa es el contrato acumulativo del proyecto. En CI añadiremos además package y runtime real.

Fallos útiles y mensajes de aserción

Un test debe fallar de forma que ayude a diagnosticar. Cuando una aserción pueda resultar ambigua, JUnit permite añadir un mensaje:

```
assertEquals(  
    "3",  
    creado.getIdentificador(),  
    "el servicio debe usar el identificador suministrado por el repositorio");
```

No necesitamos mensajes redundantes en todas las aserciones, pero sí conviene que los datos, nombres y estructura del test permitan entender con rapidez qué contrato se rompió. Una suite que sólo dice “expected true but was false” en decenas de tests cuesta mucho más mantener que una suite con escenarios bien nombrados.

Evitar tests que sólo duplican implementación

Si un test reproduce exactamente el algoritmo del método probado para calcular el resultado esperado, ambos pueden equivocarse de la misma manera. Siempre que sea posible, expresa expectativas desde el contrato. Para una creación, nos importa que haya ID, que se guarde y que la regla de duplicado se respete; no necesitamos reimplementar en el test todos los detalles internos de cómo el servicio llega a ese resultado.

Regla final de la suite

Cada test debe poder ejecutarse solo y también como parte de la suite completa, con el mismo resultado. Si un test cambia el estado global de manera que condiciona al siguiente, la suite está ocultando acoplamiento y debe corregirse antes de considerarse una red de seguridad fiable.

Punto 2.6 - Buenas prácticas de organización, calidad y documentación

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Aplicar convenciones de nomenclatura coherentes a clases, métodos, variables y tests.
2. Comparar organización por capas, por funcionalidad e híbrida.
3. Ordenar internamente una clase Java para mejorar legibilidad y mantenimiento.
4. Entender el papel de Checkstyle, SpotBugs y SonarQube.
5. Configurar Checkstyle como gate reproducible del build.
6. Documentar clases y contratos públicos con Javadoc.
7. Mantener un README útil para ejecutar y comprender el proyecto.
8. Publicar documentación OpenAPI/Swagger compatible con Spring Boot 3.5.x.
9. Evitar que documentación, calidad y organización se conviertan en tareas manuales que se olvidan.
10. Crear una utilidad común sólo cuando realmente representa comportamiento transversal reutilizable.

Bloque 1 - Convenciones de nomenclatura

1.1 Los nombres forman parte del diseño

Un nombre no es decoración. Es la primera explicación del código.

Compara:

```
public void p(AlumnoDTO a) { ... }
```

con:

```
public void promocionarAlumno(AlumnoDTO alumno) { ... }
```

El segundo reduce la carga de interpretación.

Reglas básicas que seguiremos:

Clases	-> PascalCase
métodos	-> camelCase
variables	-> camelCase
constantes	-> MAYUSCULAS_CON_GUIONES_BAJOS
paquetes	-> minúsculas

Ejemplos:

```
AlumnoService
ExpedienteRepository
buscarPorId
siguienteIdentificador
MAX_INTENTOS
es.mecd.demo.miproyecto.alumno
```

1.2 Nombres orientados a intención

Evita nombres que describen únicamente el mecanismo:

```
procesar()
hacer()
ejecutar()
getData()
```

Prefiere nombres que expliquen el propósito:

```
crearAlumno()
buscarPorDni()
cargarDatosIniciales()
formatearFecha()
```

1.3 Nombres de tests

La fuente insiste en que los tests deberían describir escenario y resultado.

Un buen nombre:

```
crearConDniDuplicadoLanzaNegocioException()
```

es mejor que:

```
testCrear2()
```

El test actúa como documentación ejecutable.

Pregunta

¿Por qué un nombre de test debe contar el escenario además del método probado?

Respuesta razonada

Porque un mismo método tiene múltiples comportamientos relevantes. El nombre debe permitir saber qué contrato ha fallado sin abrir primero toda la implementación del test.

Bloque 2 - Organización de paquetes

2.1 Organización por capas

Hasta este momento el proyecto se organiza principalmente así:

```
controller/  
service/  
repository/  
dto/  
exception/  
config/
```

Ventajas:

- sencilla de entender al principio;
- hace visibles las capas arquitectónicas;
- encaja bien con un proyecto pequeño.

Problema al crecer: los elementos de una misma funcionalidad terminan dispersos en muchas carpetas.

2.2 Organización por funcionalidad

Otra estrategia agrupa por caso de negocio:

```
alumno/  
  AlumnoController.java  
  AlumnoService.java  
  AlumnoRepository.java  
  AlumnoDTO.java  
  
expediente/  
  ExpedienteController.java  
  ExpedienteService.java  
  ExpedienteRepository.java  
  ExpedienteDTO.java
```

Ventaja: todo lo relacionado con una funcionalidad está cerca.

2.3 Organización híbrida

En proyectos reales es habitual combinar ambas ideas:

```
alumno/  
  web/  
    service/  
    repository/  
    dto/  
  
expediente/  
  web/  
    service/  
    repository/  
    dto/  
  
config/  
common/
```

No existe una única estructura universal. La decisión depende de tamaño, dominio, equipo y evolución prevista.

2.4 Qué criterio usaremos en el curso

La fuente propone reorganizar por funcionalidad al final de M2. Conservaremos el ejercicio porque obliga a comprobar que **mover paquetes no debe romper contratos externos**.

Pero evitaremos mover archivos de forma ciega. Cada cambio de paquete deberá actualizar:

- package;
- imports;
- tests;
- documentación;
- cualquier referencia de configuración;
- trazabilidad interna del curso.

Pregunta

¿Cuándo empieza a resultar incómoda una organización exclusivamente por capas?

Respuesta razonada

Cuando una funcionalidad obliga a saltar continuamente entre muchas carpetas y el número de clases de cada capa crece. En ese punto agrupar por dominio o funcionalidad puede reducir dispersión.

Bloque 3 - Estructura interna de una clase

3.1 Orden consistente

Una clase resulta más fácil de leer cuando mantiene un orden predecible. Una convención razonable es:

1. Javadoc de clase.
2. Anotaciones.
3. Constantes.
4. Campos.
5. Constructores.
6. Métodos públicos.
7. Métodos protegidos/package-private si existen.
8. Métodos privados.
9. Clases internas si fueran necesarias.

Ejemplo:

```
/** Servicio de operaciones de alumnos. */
@Service
public class AlumnoService {

    private final AlumnoRepository repositorio;

    public AlumnoService(AlumnoRepository repositorio) {
        this.repositorio = repositorio;
    }

    public Optional<AlumnoDTO> consultar(String id) {
        return repositorio.buscarPorId(id);
    }

    private void validar(...) {
        // ...
    }
}
```

```
}  
}
```

3.2 Tamaño y responsabilidad

Un método excesivamente largo suele ocultar varias operaciones distintas. Antes de fragmentarlo mecánicamente, pregunta:

¿hay más de una responsabilidad?
¿hay una regla con nombre propio?
¿hay código repetido?
¿hay una operación que merezca test independiente?

3.3 Visibilidad mínima necesaria

No todo debe ser public.

Regla útil:

public -> forma parte del contrato externo de la clase
private -> detalle interno
package-private/protected -> sólo si existe una razón de diseño

3.4 Repositorio hoy y en Spring Data JPA

La fuente anticipa una pregunta importante: en M4 un repositorio Spring Data se declarará normalmente como interfaz.

Ejemplo futuro:

```
public interface AlumnoRepository  
    extends JpaRepository<Alumno, Long> {  
}
```

No lo adelantamos ahora en el proyecto funcional. En M2 usamos clases concretas porque todavía estamos aprendiendo la separación sin JPA.

Pregunta

¿Por qué no convertimos ya nuestros repositorios en interfaces de JpaRepository?

Respuesta razonada

Porque introduciríamos JPA, entidades y persistencia real antes de haberlos explicado. El curso separa deliberadamente arquitectura y tecnología de persistencia.

Bloque 4 - Herramientas de calidad

4.1 Checkstyle

Checkstyle verifica reglas de estilo y estructura del código Java.

Puede detectar, por ejemplo:

- nombres que incumplen convenciones;
- longitud de línea;
- imports desordenados o no permitidos;
- llaves e indentación;
- ciertos requisitos de Javadoc.

En Maven se integra mediante maven-checkstyle-plugin.

Para la edición 2026 utilizaremos la versión estable **3.6.0** del plugin, no las versiones antiguas de la fuente.

Ejemplo:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-checkstyle-plugin</artifactId>
  <version>3.6.0</version>
</plugin>
```

La versión del plugin y la versión interna de Checkstyle no son el mismo concepto; el plugin resuelve su propia dependencia de Checkstyle según configuración.

4.2 SpotBugs

SpotBugs analiza bytecode buscando patrones asociados a errores potenciales:

- posibles NullPointerException;
- comparaciones sospechosas;
- recursos mal gestionados;
- problemas de concurrencia;
- errores habituales de API.

No reemplaza tests ni revisión humana. Es otra señal automática.

4.3 SonarQube

SonarQube agrega métricas y análisis más amplios:

- bugs;
- vulnerabilidades;
- code smells;
- duplicación;
- cobertura;
- deuda técnica.

En este punto lo estudiaremos como herramienta del ecosistema; no necesitamos convertir el curso en una instalación completa de servidor SonarQube para entender su papel.

4.4 Calidad en CI

La gran ventaja de integrar herramientas en el build es que una regla deja de depender de que alguien “se acuerde”.

código -> build -> checks -> resultado reproducible

Un check demasiado estricto o mal elegido también puede perjudicar. Por eso las reglas deben estar justificadas y adaptadas al nivel del proyecto.

Pregunta

¿Por qué no basta con ejecutar Checkstyle manualmente una vez antes de entregar?

Respuesta razonada

Porque el código seguirá cambiando. Integrarlo en el build convierte la regla en una garantía repetible para cada cambio futuro.

Bloque 5 - Documentación: Javadoc, README y OpenAPI

5.1 Javadoc

Javadoc documenta contratos públicos de Java.

Ejemplo de clase:

```
/**
 * Servicio que aplica las reglas de negocio de alumnos.
 */
@Service
public class AlumnoService {
}
```

Ejemplo de método:

```
/**
 * Consulta un alumno por su identificador.
 *
 * @param id identificador del alumno
 * @return alumno encontrado o un Optional vacío
 */
public Optional<AlumnoDTO> consultar(String id) {
    return repositorio.buscarPorId(id);
}
```

Cuando un método puede lanzar una excepción relevante:

```
/**
 * Crea un alumno.
 *
 * @param dto datos del alumno
 * @return alumno almacenado
 * @throws NegocioException si el DNI ya existe
 */
```

Javadoc debe explicar el contrato, no repetir línea por línea la implementación.

5.2 Generar documentación Java

Maven puede generar HTML:

```
./mvnw javadoc:javadoc
```

La salida habitual queda bajo:

```
target/site/apidocs/
```

La documentación sólo es útil si compila y no contiene referencias obsoletas.

5.3 README

Un README útil debería permitir a otra persona responder rápidamente:

- ¿qué hace el proyecto?;
- ¿qué Java necesita?;
- ¿cómo se compila?;
- ¿cómo se ejecuta?;
- ¿cómo se ejecutan los tests?;
- ¿qué endpoints principales existen?;
- ¿qué perfiles/configuración hay?;
- ¿dónde está la documentación OpenAPI?;
- ¿qué decisiones arquitectónicas importantes se han tomado?

5.4 OpenAPI

OpenAPI describe una API de forma estructurada. Swagger UI utiliza esa descripción para proporcionar documentación interactiva.

Para Spring Boot 3.5.x, la matriz oficial de springdoc indica utilizar la línea **2.8.x**. No utilizaremos 2.3.0 de la fuente, que corresponde a líneas anteriores de Spring Boot.

Dependencia:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>${springdoc.version}</version>
</dependency>
```

Y en propiedades podemos fijar una versión 2.8.x probada por CI.

Rutas habituales:

```
/v3/api-docs
/swagger-ui.html
```

5.5 Enriquecer el contrato

Podemos añadir anotaciones:

```
@Operation(summary = "Consulta un alumno por ID")
@ApiResponses({
  @ApiResponse(responseCode = "200", description = "Alumno encontrado"),
  @ApiResponse(responseCode = "404", description = "Alumno inexistente")
})
```

No conviene duplicar toda la documentación manualmente: OpenAPI debe complementar el código, no convertirse en otra fuente imposible de mantener.

Pregunta

¿Por qué merece la pena documentar también errores 4xx en OpenAPI?

Respuesta razonada

Porque forman parte del contrato que consume el cliente. Saber sólo cómo es una respuesta 200 no basta para integrar correctamente una API.

Resumen del punto 2.6

La calidad final del proyecto no depende sólo de que “funcione”. También depende de que otro desarrollador pueda:

- encontrar el código
- entender nombres y responsabilidades
- compilarlo
- probarlo
- leer su documentación
- inspeccionar la API
- recibir feedback automático de calidad

M2 termina por tanto conectando arquitectura, testing, configuración, documentación y herramientas de calidad en un único proyecto acumulativo.

Profundización: paquetes, comentarios y análisis estático

Convenciones de paquetes

Los paquetes se escriben en minúsculas y, en Java, suelen comenzar por un dominio invertido o espacio de nombres estable:

```
es.mecd.demo.miproyecto
```

A partir de ahí añadimos nombres que expresen estructura o funcionalidad:

```
es.mecd.demo.miproyecto.alumno  
es.mecd.demo.miproyecto.expediente  
es.mecd.demo.miproyecto.common.util
```

Evita:

```
Utils2  
MisCosas  
temporal  
nuevo  
pruebasfinales
```

porque describen circunstancias momentáneas, no responsabilidad estable.

Buenas prácticas específicas en controladores

Un controlador debería:

- ser pequeño;
- delegar en servicios;
- trabajar con DTOs y tipos de la frontera web;
- decidir status y cabeceras;
- no manipular directamente repositorios;
- no contener consultas de datos ni reglas complejas;
- mantener rutas coherentes.

Mal ejemplo:

```
@PostMapping  
public ResponseEntity<?> crear(@RequestBody AlumnoDTO dto) {  
    if (repositorio.existePorDni(dto.getDni())) {  
        // regla + datos + HTTP mezclados  
    }  
}
```

La arquitectura que hemos construido evita ese acoplamiento.

Buenas prácticas específicas en servicios

Un servicio debería:

- expresar casos de uso y reglas;
- depender de repositorios por constructor;
- no devolver ResponseEntity;
- evitar detalles de almacenamiento;
- lanzar excepciones de negocio cuando el dominio lo requiera;
- ser fácil de probar sin arrancar Spring.

Buenas prácticas específicas en repositorios

Un repositorio debería:

- ofrecer operaciones de acceso a datos;

- no contener reglas de negocio;
- proteger su estado interno;
- usar `Optional` cuando un resultado único puede faltar;
- no devolver `null` como resultado normal;
- no conocer HTTP;
- encapsular la tecnología de almacenamiento.

Comentarios en el código

La fuente dedica un apartado específico a los comentarios. Un comentario es útil cuando explica una decisión o restricción no evidente.

Útil:

```
// Conservamos una secuencia monotonica para evitar reutilizar IDs tras DELETE.
```

Poco útil:

```
// Incrementamos secuencia  
secuencia.incrementAndGet();
```

El código debe explicar el **qué** mediante nombres. El comentario debería aportar el **por qué** cuando éste no sea evidente.

Evita comentarios obsoletos. Un comentario incorrecto es peor que no tener comentario.

SpotBugs con más detalle

SpotBugs analiza bytecode y puede integrarse en Maven. Un ejemplo típico de plugin es:

```
<plugin>  
  <groupId>com.github.spotbugs</groupId>  
  <artifactId>spotbugs-maven-plugin</artifactId>  
</plugin>
```

En el curso no fijaremos un gate obligatorio de SpotBugs hasta probar su configuración sobre todo el código, pero sí debemos entender qué aporta: detectar patrones sospechosos que un compilador Java válido permite.

Ejemplos de categorías:

```
correctness  
bad practice  
performance  
multithreaded correctness  
malicious code vulnerability
```

SonarQube, cobertura y deuda técnica

SonarQube puede combinar análisis estático y métricas históricas. Conceptos habituales:

- *bug*: patrón que probablemente produce comportamiento incorrecto;
- *vulnerability*: problema de seguridad;
- *code smell*: señal de mantenibilidad;
- *coverage*: proporción de código ejecutado por tests según la métrica utilizada;
- *duplication*: código repetido;
- *technical debt*: estimación del esfuerzo de corrección.

Una puntuación alta no sustituye criterio de ingeniería. El objetivo es usar las métricas como señales, no como objetivos aislados que incentiven “jugar con el número”.

Herramientas en CI

Podemos imaginar una pipeline final:

```
compilar
-> tests
-> Checkstyle
-> análisis estático
-> package
-> runtime smoke test
```

Un cambio sólo se considera publicable cuando pasa el conjunto de gates definido por el proyecto.

Convenciones concretas para tipos Java

La fuente enumera convenciones que debemos conservar.

Clases e interfaces:

```
AlumnoService
AlumnoRepository
FechaFormatter
```

Métodos y variables:

```
buscarPorId
fechaSolicitud
alumnoCreado
```

Constantes:

```
private static final int MAX_REINTENTOS = 3;
```

Booleanos suelen leerse mejor como predicados:

```
activo
esValido
tieneDocumentos
```

Evita abreviaturas que sólo entiende quien escribió el código:

```
svc
repo2
dtoX
x1
```

salvo convenciones locales muy conocidas y de alcance mínimo.

Organización híbrida y paquetes comunes

Un paquete `common` no debe convertirse en el lugar donde se arroja cualquier clase difícil de clasificar.

Correcto:

```
common/exception
common/util
```

si realmente contiene elementos transversales usados por varias funcionalidades.

Sospechoso:

```
common/misc
common/others
common/temp
```

Si una clase sólo pertenece a alumnos, debe permanecer cerca de alumnos.

Estructura interna: ejemplo completo

```
/** Servicio de alumnos. */
@Service
public class AlumnoService {

    private static final String SUFIJO_PROMOCION = " (promocionado)";

    private final AlumnoRepository repositorio;

    public AlumnoService(AlumnoRepository repositorio) {
        this.repositorio = repositorio;
    }

    public Optional<AlumnoDTO> consultar(String id) {
        return repositorio.buscarPorId(id);
    }

    public void promocionar() {
        repositorio.listarTodos().forEach(this::promocionarYGuardar);
    }

    private void promocionarYGuardar(AlumnoDTO alumno) {
        alumno.setCurso(alumno.getCurso() + SUFIJO_PROMOCION);
        repositorio.guardar(alumno);
    }
}
```

La intención no es imponer este ejemplo exacto, sino mostrar que constantes, campos, constructor, API pública y helpers privados aparecen en un orden predecible.

Comentarios de decisión

Un buen comentario registra una decisión que no puede deducirse fácilmente:

```
// Ordenamos por ID para que la respuesta observable sea determinista aunque
// ConcurrentHashMap no preserve orden de iteración.
```

Ese comentario tiene valor porque explica una razón. En cambio:

```
// Obtenemos la lista
var lista = repositorio.listarTodos();
```

no aporta información.

Configuraciones predefinidas de Checkstyle

Checkstyle incluye configuraciones conocidas como `google_checks.xml` o `sun_checks.xml`. Son útiles como referencia, pero aplicar una configuración completa sin adaptación puede producir cientos de violaciones que distraen del objetivo pedagógico.

Nuestro enfoque es comenzar con un conjunto pequeño y comprensible y endurecerlo sólo cuando exista una razón clara.

Checkstyle no es un formateador

Checkstyle principalmente **verifica** reglas; no debemos confundirlo con herramientas de formateo automático. Un equipo puede combinar un formateador y Checkstyle, pero cumplen funciones diferentes.

SpotBugs y falsos positivos

Un analizador estático puede señalar código que requiere revisión pero que, en contexto, no es un error. La respuesta correcta no es ignorar automáticamente ni obedecer ciegamente: hay que evaluar la advertencia y, si se suprime, documentar el motivo.

Cobertura no equivale a calidad

Un proyecto puede tener 100 % de cobertura ejecutando líneas sin verificar comportamiento útil. Preferimos tests que expresen contratos relevantes aunque la cifra global no sea máxima.

Ejemplo de test poco valioso:

```
@Test
void llamaMetodo() {
    servicio.consultar("1");
}
```

sin aserción ni verificación.

Ejemplo útil:

```
assertTrue(servicio.consultar("1").isPresent());
```

con el escenario controlado.

README como interfaz del proyecto

El README no es una memoria histórica. Debe reflejar el estado ejecutable actual. Cuando cambia:

- versión de Java
- comando de arranque
- perfil
- endpoint
- ruta Swagger

debe actualizarse el documento correspondiente.

Una instrucción que ya no funciona erosiona rápidamente la confianza en toda la documentación.

Documentación generada y documentación escrita

OpenAPI y Javadoc pueden generar documentación a partir del código, pero no eliminan la necesidad de un README. Cada herramienta responde a una pregunta distinta:

- Javadoc -> ¿qué contrato ofrecen estas clases Java?
- OpenAPI -> ¿qué contrato HTTP ofrece esta aplicación?
- README -> ¿qué es este proyecto y cómo lo ejecuto?

Mantener las tres capas evita intentar resolver todos los públicos con un único documento.

Calidad como conjunto de señales

Ninguna herramienta individual puede certificar que el software es correcto. Una entrega razonable combina:

- compilador
- + tests
- + análisis de estilo
- + análisis estático
- + revisión de arquitectura
- + runtime real
- + documentación ejecutable

La fuerza está en la combinación. Checkstyle puede aprobar código lógicamente incorrecto; los tests pueden pasar con nombres ilegibles; OpenAPI puede describir perfectamente una API mal diseñada. Por eso M2 termina con varias barreras complementarias.

Definición de terminado para calidad

En este curso una mejora de organización no se considera terminada porque “el IDE ya no muestra errores”. El cierre exige que el build reproducible siga funcionando, los tests permanezcan verdes, el Javadoc pueda generarse, Checkstyle pase con la configuración publicada, el JAR arranque y la documentación OpenAPI exponga los endpoints acumulativos. Esa definición evita que una refactorización puramente estética rompa comportamiento o documentación sin que nadie lo detecte.

Automatizar sin convertir la herramienta en el objetivo

El propósito de Checkstyle, Javadoc u OpenAPI no es “tener otra casilla verde”. Cada herramienta debe reducir un riesgo concreto: inconsistencia de estilo, contratos Java poco claros o API difícil de integrar. Si una regla automática genera ruido permanente y nadie entiende qué riesgo evita, debe revisarse. Calidad significa feedback útil y sostenible, no acumulación indiscriminada de plugins.